Action-Conditioned Predictive Consistency for Robust Visual World Models

May 30, 2026

Abstract

Latent predictive world models such as Joint-Embedding Predictive Architectures (JEPAs) predict future representations rather than pixels, but this does not by itself define visual robustness for control. We argue that robustness should be measured as *action-conditioned predictive consistency*: clean and corrupted observations of the same underlying state may encode differently, yet under the same history and action sequence they should induce consistent task-relevant future predictions. This consistency must be selective, preserving state differences that change transitions, costs, or optimal behaviour.

We study this requirement with controlled Gaussian pixel corruptions on **LeWorldModel (LeWM)** [25] across PushT, TwoRoom, Reacher, and Cube: 36 checkpoints, 3 seeds, and 100 trajectories per seed. Without noise-aware training, PushT drops from 86.33% to 4.67% under pixels+goal noise with standard deviation 0.08, and TwoRoom drops from 94.00% to 50.00%. Noise augmentation recovers much of this performance, but only as a coarse global scalar pressure with broad, task-dependent plateaus. Diagnostics show why encoder-level or pointwise accounts are incomplete: after controlling for train-time noise, the strongest single-step fragility metric does not explain the PushT corruption gap (partial Spearman $\rho = +0.06$), whereas multi-step predictor drift retains residual signal on Reacher ($\rho = +0.79$). A PLDM replication preserves the task-level signature, and a negative heteroscedastic-reweighting result shows that "hard" transitions can be action-relevant rather than nuisance variation. These findings motivate measuring and regularising consistency *after* action-conditioned prediction while preserving discriminability; they do not yet constitute a new training algorithm.

Keywords: visual world models; JEPA; action-conditioned predictive consistency; visual robustness; predictive-dynamics diagnostics.

1 Introduction

1.1 Latent prediction is not a robustness definition

Since Yann LeCun proposed the Joint-Embedding Predictive Architecture (JEPA) [23], the paradigm has been advanced as a direction for self-supervised learning. Unlike generative models (variational autoencoders, diffusion), a JEPA does not reconstruct pixels; it predicts future *representations* in latent space. A common motivating intuition is that latent prediction reduces pressure to preserve observation-level detail that is irrelevant to predicting future target representations, and may therefore encourage more abstract representations [2, 4]. Recent theory makes a sharper version of this point: relative to reconstruction, joint-embedding prediction lowers the burden of encoding high-magnitude irrelevant features, yet it still needs augmentation or an inductive bias to identify *which* variations are irrelevant [37]. None of this is a robustness guarantee for control, where the notion of "irrelevant" is action- and transition-dependent rather than purely visual.

For a visual world model used in closed-loop control, the relevant question is therefore not whether an unperturbed observation o and its corrupted counterpart $\tilde{o}$ have identical latents, but whether they induce *consistent predictions under the same action intervention.* A linear probe or a recognition test can be satisfied by encoder-level invariance; planning cannot. The model must suppress nuisance visual variation *while preserving the action-relevant distinctions that change which action is best* — and it is the model's *predictions*, not its raw encodings, that feed the planner.

The empirical starting point is that latent prediction does not supply this for free. On PushT (2D pushing), the LeWM checkpoint trained without input-noise augmentation achieves 86.33% on unperturbed evaluation images but falls to 4.67% under Gaussian pixel noise with standard deviation (std) = 0.08 applied to both observation and goal — near-random. TwoRoom (2D navigation) drops from 94.00% to 50.00%. Visual-corruption fragility is itself well documented in pixel-based reinforcement learning (RL) [41, 40, 17] and visual model-based RL [26]; we do not claim it as a new phenomenon. We use it here as a *controlled probe* of a JEPA latent world model under closed-loop control, where actions are optimised with the Cross-Entropy Method (CEM) as an evaluation-time implementation detail rather than a part of our thesis.

1.2 From visual invariance to action-conditioned predictive consistency

We reframe visual robustness for world-model control at the level of *action-conditioned predictive dynamics.* Let $z = E(h)$ and $\tilde{z} = E(\tilde{h})$ be the latents an encoder E produces from an unperturbed history h and a corrupted history $\tilde{h}$ of the same underlying state, and let $F^k(\cdot, \mathbf{a}_{0:k-1})$ be the action-conditioned latent predictor rolled out for k steps under an action sequence $\mathbf{a}$. We *allow* $z \neq \tilde{z}$. The robustness target is placed *after* the predictor: in task-relevant coordinates,

$$F^k(z, \mathbf{a}_{0:k-1}) \;\approx\; F^k(\tilde{z}, \mathbf{a}_{0:k-1}), \qquad k = 1, \dots, H, \tag{1}$$

so that the two branches induce the same next-state and rollout predictions under the same action intervention. Crucially this is *selective*: it should hold only for nuisance perturbations of the same underlying state. State differences that change action-conditioned transitions, costs, or the optimal action must remain *discriminable*, or the planner loses the distinctions it needs. We make both halves precise in Section 3.

This reframing changes what counts as a robustness measurement. Encoder-level latent closeness, $\|z - \tilde{z}\| \to 0$, is neither necessary (a large but task-irrelevant encoder shift can wash out after the predictor) nor sufficient (a small encoder shift can still flip the planned action). Planning, cost, and action metrics remain useful, but as *downstream readouts* of predictive consistency rather than as the central definition; the object we ultimately want to regularise is the action-conditioned predictive dynamics.

1.3 Existing corruption results as a controlled probe

A natural remedy is input-side noise augmentation during training, long validated in supervised and contrastive learning [6, 7]. We use a systematic sweep across four tasks at eight levels of `std_max` $\in \{0.01, \dots, 0.08\}$ as a probe of how a single global pressure exposes the selective-consistency tension between contracting nuisance variation and preserving action-relevant discriminability:

- **TwoRoom** (visually redundant navigation): unperturbed success rises monotonically with noise, peaking at `std_max` = 0.08 (98.33% / 98.67%).
- **PushT** (contact-heavy manipulation): unperturbed success peaks at `std_max` = 0.03 (89.67%), but robustness at px+goal 0.08 peaks at `std_max` = 0.06 (87.00%) — unperturbed and corrupted-evaluation optima dissociate.

- **Reacher** (continuous reaching): lies on a 0.02–0.06 plateau; the unperturbed point-best is at `std_max` = 0.06 (86.00%), while px+goal 0.08 point-best is at `std_max` = 0.02 (85.67%). Very low noise (0.01) is statistically indistinguishable from base.
- **Cube** (structured 3D manipulation): noise sweep is the weakest of the four, with no monotone trend on unperturbed evaluation. We read this as the mildest manifestation of the selective-consistency tension rather than an absence: a structured action sequence and predictable visual-action coupling supply enough robustness that input-side augmentation buys comparatively little.

Read together, these are not the signature of a single jointly optimal noise level; input-side Gaussian augmentation behaves as a *coarse global scalar pressure* with broad, task-dependent plateaus. It cannot distinguish "background visual redundancy that should be made invariant" from "control-relevant features that should retain resolution" — exactly the selectivity an action-conditioned consistency objective is meant to supply. We therefore treat the sweep, the diagnostics, and the negative results below as evidence that *localises mechanisms and motivates a method target*, not as the paper's central contribution.

1.4 Contributions

C1 — Problem reframing. We argue that visual robustness for world-model control should be formulated as *action-conditioned predictive consistency*, together with an action-relevant discriminability countercondition, rather than as encoder-level latent invariance. Section 3 makes the definition precise and separates it from the downstream planning, cost, and action readouts that test whether predictive mismatch actually matters for control.

C2 — Diagnostic evidence. Using controlled Gaussian-corruption experiments on JEPA world-model control — 4 tasks × 9 configurations (one no-noise baseline plus eight noise-augmented levels), all 36 checkpoints evaluated under a unified 3-seed × 100-trajectory protocol (300 trajectories per condition) and replicated on a second method family, Planning with Latent Dynamics Models (PLDM) — we establish three points. (i) Visual perturbations cause severe closed-loop failure, with a task-dependent cliff-and-recovery pattern that is not tied to a single LeWM checkpoint family (Section F). (ii) Input-side noise augmentation recovers performance but only as a coarse global pressure. (iii) Pointwise, single-step fragility metrics are insufficient: after conditioning on `std_max`, the strongest single-step fragility ratio does not explain the unperturbed-to-corrupted gap (partial Spearman $\rho = +0.06$ on PushT, 95% bootstrap confidence interval (CI) $[-0.00, +0.25]$; the null replicates on PLDM and on the joint $n = 18$ sample), whereas multi-step predictor drift retains residual signal on some tasks (Reacher partial $\rho = +0.79$) (Section 5.5).

C3 — Selective-consistency diagnostics. We define a family of action-conditioned consistency diagnostics that compare unperturbed and corrupted predictions under the *same* action sequence while separately measuring action-relevant discriminability (Section 3.2). They sharpen the existing single-step fragility ratio and multi-step rollout drift into paired, action-conditioned quantities and specify the diagnostic pass needed before claiming an ACPC training method.

C4 — Method-design implication. The evidence motivates *adaptive predictive-dynamics consistency*: enforce consistency for same-state perturbations after the action-conditioned predictor while preserving distinctions for action-sensitive transitions, gated by action/transition sensitivity rather than by prediction error. A negative heteroscedastic-reweighting result ("hard ≠ nuisance";

Sections D and 6.6) shows why error-based downweighting is the wrong gate. Because we do not yet report a method experiment, we state this as a design implication and future direction.

1.5 Organisation

Section 2 reviews related work; Section 3 defines action-conditioned predictive consistency, the discriminability countercondition, and the proposed consistency diagnostics; Section 4 defines the LeWM background, the noise protocol, and the existing five-layer diagnostic framework; Section 5 reports the experimental findings as a probe; Section 6 discusses mechanism and method implications; Section 7 concludes.

2 Related Work

2.1 JEPA and latent world models

JEPA [23] predicts in latent space rather than reconstructing pixels. I-JEPA [2] predicts target representations from a masked context; V-JEPA [4, 3] extends the framework to video understanding and video-driven world modelling; LeWM [25] is the first end-to-end stable JEPA world model, using **SIGReg** (Sketch Isotropic Gaussian Regularizer) — random projections plus Epps–Pulley empirical-characteristic-function matching [9] — to prevent representation collapse without batch normalisation. LeWM is our baseline. The original LeWM paper reports a Violation-of-Expectation (VoE) experiment showing the model is sensitive to physical perturbations (object teleportation) but not to visual perturbations (colour change). VoE measures prediction error (surprise), not control success rate, and colour change differs from pixel-level Gaussian noise. We therefore focus on success-rate robustness under controlled test-time pixel corruptions.

For a second method family we evaluate PLDM [31, 32] as implemented in `stable-worldmodel` [24] on the full sweep grid (Section F). The PLDM sweep covers the same four tasks and the same eight values of `std_max` as our LeWM sweep, under the same 3-seed $\times$ 100-trajectory evaluation protocol. We use PLDM to test which findings replicate across method families and which are LeWM-specific; the predictive-consistency framing and the diagnostic toolkit themselves are introduced and analysed on LeWM, and the LeWM canonical tables in the main text are kept method-pure for clarity.

2.2 Joint-embedding, reconstruction, and augmentation alignment

Whether a representation discards nuisance variation depends on the training signal, not on latent prediction alone. Wang and Isola [39] decompose the contrastive loss into *alignment* (positive pairs close) and *uniformity* (features spread on the hypersphere); Tamkin et al. [34] argue that label-destroying augmentations act as feature dropout rather than pure invariance inducers; and Zhang and Ma [43] note that augmentation modules impose invariances that must be matched to the task. Van Assel et al. [37] make the comparison precise for latent-space prediction: relative to reconstruction, joint-embedding prediction lowers the burden of encoding high-magnitude irrelevant features, but it still relies on augmentation or an inductive bias to identify *which* variations are irrelevant. This is exactly the gap we target. In control, "irrelevant" is not an image-level or label-level property but an action- and transition-conditioned one, so latent prediction by itself does not tell the model which visual variations to contract. Concurrent seq-JEPA [13] resolves a related *invariance–equivariance* tension architecturally; we instead place the requirement on the model's predictions rather than its encodings.

2.3 LeJEPA identifiability and latent world-model theory

A complementary theoretical line studies when a learned representation recovers the latent structure of the world. Klindt et al. [20] analyse when LeJEPA — alignment plus Gaussian regularization — recovers the world's latent variables (for instance under Gaussian latent structure and stationary additive-noise transitions) and thereby supports latent-space planning, while noting that action-conditioned transitions remain a natural extension. Our question is *complementary*, and we do *not* claim to prove or strengthen any identifiability result: given a learned visual world model, we ask when unperturbed and visually corrupted observations of the same underlying state induce *consistent action-conditioned predictions*, and when corruptions instead push the model into a different predictive neighbourhood. Identifiability concerns the clean, state-side representation; predictive consistency concerns the predictor's behaviour under corrupted observations.

2.4 Bisimulation and control-relevant representations

Bisimulation gives a principled notion of control-relevant state equivalence: states with the same reward and transition behaviour can be merged. DeepMDP [12] learns latent models with transition- and reward-matching losses; Deep Bisimulation for Control (DBC) [42] learns distractor-robust representations by making the latent metric track a bisimulation metric; bisimulation-regularized MPC encoders extend this idea to planning for noise robustness [30]; and the contemporaneous Bisim-JEPA [36] adds a bisimulation encoder to a JEPA-style visual world model, mapping dynamically similar states to nearby latents to suppress slow-feature distractors. Our framing differs in substance, not by stacking conditions. (i) We study *paired* unperturbed/corrupted observations of the *same* state under the *same* action sequence, rather than learning a single invariant state metric. (ii) We do *not* require the corrupted latent to collapse onto the clean one — $z \neq \tilde{z}$ is allowed — and ask for agreement only after the action-conditioned predictor, in task-relevant coordinates. (iii) We keep the discriminability guard explicit, so that action-sensitive transitions (such as contact events) are preserved rather than absorbed into an equivalence class. A method that merely learned a more invariant embedding would fall back into the bisimulation family; the action-conditioned, sensitivity-gated consistency is what separates the target.

2.5 Visual robustness in model-based and pixel-based RL

Input-side augmentation is an established route to visual robustness in pixel-based RL: DrQ [41] and DrQ-v2 [40] regularise a model-free critic with random-shift augmentation, and SODA / DeepMind Control Generalization Benchmark (DMC-GB) [17] formalises a visual-generalisation benchmark with random colours and video backgrounds. These study model-free policies with explicit reward signals, not latent-prediction world models. Among world models, DreamerV3 [15] and TD-MPC2 [16] learn visual encoders and latent dynamics that may tolerate benign variation but do not by themselves settle sensor-noise robustness. ViGMO [26] is closest in spirit: it studies zero-shot visual generalization in model-based RL under sensor noise (Gaussian noise and blur, the two families we use as a probe) and proposes a latent-consistency loss, reporting that generalization baselines degrade sharply with latent-consistency strongest among them. The boundary we draw is specific: ViGMO enforces *generic* latent consistency at the encoder, whereas our target is consistency of *action-conditioned* one-step and multi-step rollouts between paired unperturbed/corrupted branches, plus an explicit discriminability guard so that consistency is not achieved by collapsing action-relevant transitions. JEPA-side robustness studies — diffusion-noise augmentation (N-JEPA [27]), a synthetic Noisy-TV distractor where JEPA models keep signal-recovery $R^2 > 0.84$ (VJEPA [18]), and low-SNR ultrasound (US-JEPA [28]) — show the topic is active; VJEPA's positive result is not contradicted by ours, because a signal-recovery probe need only preserve recoverable information, whereas closed-loop control demands predictions that support action optimisation. Finally, `stable-worldmodel` [24] establishes controllable visual and physical variations as part of the world-model evaluation ecosystem, which is why we present visual perturbation as a controlled probe rather than as a novel benchmark.

2.6 Value-aware and value-equivalent model learning

A separate principle holds that a model need not fit full state-to-state dynamics; it should serve its downstream use. Value-equivalent models [14] produce identical Bellman updates for a chosen set of policies and value functions, and value-aware model learning [38] shapes the model loss by its downstream value estimation. We adopt this downstream-consequence principle but specialise it to visual robustness and predictive dynamics: two observations of the same state should remain equivalent *after* action-conditioned prediction, while genuinely different transitions stay distinguishable. VIBR [8] makes a related point on the model-free side — a view-invariant *value function* can be more appropriate than an invariant representation — which supports our claim that robustness should not stop at representation closeness; the difference is that we target the action-conditioned predictive dynamics of a world model rather than a value function, and we do not collapse the world model to a value-equivalence class because contact-sensitive predictive detail must be retained.

2.7 Representation diagnostics and collapse analysis

Self-supervised learning broadly uses effective rank [29], condition number, and participation ratio to diagnose dimensional collapse [19]. Next-Latent Prediction [35] uses effective latent rank to assess world-model compactness. VICReg [5] provides an anti-collapse regularizer distinct from SIGReg. Centered Kernel Alignment (CKA) [21] compares representations across networks or training conditions; we use it to compare unperturbed-input and noisy-input latents within a checkpoint. Linear probes for representation analysis follow Alain and Bengio [1]. Most relevantly, RankMe [11] showed that a label-free effective-rank diagnostic correlates with downstream linear-probe transfer across hyperparameter sweeps in self-supervised learning. Our cross-checkpoint correlation analysis (Section 5.5) asks the analogous question for control: *does a label-free predictor-sensitivity diagnostic correlate with downstream task success after the sweep-level training trend is removed?* Individual metrics are not new; our contribution is to combine them into a coherent five-layer protocol with per-token noise sensitivity, cross-checkpoint correlation, and a partial-correlation validation scheme conditioning on training noise — and to delineate where that programme succeeds and where it does not.

3 Action-Conditioned Predictive Consistency

This section defines the object on which we argue visual robustness should be measured. The thesis is that, for world-model control, robustness should be defined at the level of *action-conditioned predictive dynamics* rather than encoder-level latent invariance: unperturbed and corrupted views of the same underlying state may encode differently, but under the same history and action intervention they should induce consistent next-state and rollout predictions in task-relevant coordinates, while state differences that change action-conditioned transitions, costs, or optimal behaviour must remain distinguishable. The diagnostics in Section 3.2 operationalise this thesis and distinguish quantities already used in the present analysis from paired ACPC probes that remain to be computed.

3.1 Setup and definitions

Let o_t be an unperturbed observation and $\tilde{o}_t = \tau(o_t)$ a visually corrupted view of the *same* underlying state (τ is additive Gaussian pixel noise here, but the definitions are corruption-agnostic). Let h_t and $\tilde{h}_t$ be the corresponding observation–action histories, E_θ the encoder, and F_θ the action-conditioned latent predictor. Fix an action sequence $\mathbf{a} = a_{t:t+H-1}$ that is *identical* on both branches, and write

$$z_t = E_\theta(h_t), \quad \tilde{z}_t = E_\theta(\tilde{h}_t), \qquad \hat{z}_{t+k} = F_\theta^k(z_t, \mathbf{a}_{0:k-1}), \quad \hat{\tilde{z}}_{t+k} = F_\theta^k(\tilde{z}_t, \mathbf{a}_{0:k-1}). \tag{2}$$

Let Π denote a *task-relevant predictive readout* — the full latent, a transition delta, goal-distance or cost features, or a learned projection — and let d be a distance on its range.

Action-conditioned predictive consistency. For a same-state perturbation pair under the shared action sequence,

$$\mathrm{ACPC}_H(o_t, \tilde{o}_t, \mathbf{a}) = \sum_{k=1}^{H} \alpha_k \, d\big(\Pi(\hat{z}_{t+k}),\, \Pi(\hat{\tilde{z}}_{t+k})\big), \qquad \alpha_k \geq 0. \tag{3}$$

Robustness is the requirement that ACPC_H be *small* — crucially *not* that the encoder distance $d(z_t, \tilde{z}_t)$ be small. Encoder closeness is neither necessary (a large but task-irrelevant encoder shift can wash out through F_θ and Π) nor sufficient (a small encoder shift can still change the predicted future enough to flip the planned action).

Action-relevant discriminability (countercondition). Consistency must be *selective*. Let $Y^H(s, \mathbf{a})$ denote a task-relevant future readout that is external to the learned predictor — simulator state, cost-to-go, inverse-dynamics label, contact/keyframe tag, or another diagnostic proxy available in the dataset. For two genuinely different states s_i, s_j (with latents z_i, z_j) that admit an action sequence under which this external future readout diverges,

$$\Delta_{\mathrm{dyn}}^H(s_i, s_j, \mathbf{a}) = d_Y\big(Y^H(s_i, \mathbf{a}),\, Y^H(s_j, \mathbf{a})\big) > m, \tag{4}$$

the representation and predictions must keep them separable,

$$d\big(\Psi(z_i),\, \Psi(z_j)\big) > m', \tag{5}$$

where Ψ is a discriminability readout (latent, predicted delta, inverse-dynamics feature, cost feature, or rollout embedding) and $m, m' > 0$ are margins. Equations (3)–(5) make the central tension precise: it is not invariance versus resolution in the abstract, but

predictive consistency high for same-state visual-perturbation pairs; predictive discriminability high for state/action/transition-distinct pairs.

A method that drove $\mathrm{ACPC}_H \to 0$ by collapsing the latent would violate (5), which is why the two conditions are stated together.

3.2 Operational consistency diagnostics

Status. The list below separates descriptive quantities used in the present paper from paired ACPC quantities that define the next diagnostic pass. The paired quantities should be computed on the LeWM and PLDM checkpoints using fixed unperturbed/corrupted observation pairs and a candidate action set held identical across branches, then correlated with corrupted success and the corruption gap (see Section 6.6). ϵ below is the small constant of Equation (7).

Encoder perturbation difference (EPD), descriptive.

EPD $= d(E(o), E(\tilde{o}))$. This is the Layer-1 encoder shift of Section 4.3, retained only as a *descriptive* quantity: a large EPD is not in itself bad and a small EPD is not in itself good. It reports whether a perturbation enters the latent, not whether it matters for control.

One-step consistency (ACPC-1), paired probe.

$\mathrm{ACPC}_1 = d\big(F(E(o), a), F(E(\tilde{o}), a)\big)$, optionally normalised by a natural transition magnitude, $\mathrm{ACPC}_1/\big(d(z_t, z_{t+1}) + \epsilon\big)$, so that it is comparable across checkpoints.

Multi-step consistency (ACPC-H), paired probe.

Equation (3) with $H \in \{4, 8\}$. This is the principled, *paired* version of the existing multi-step rollout drift (`predictor_rollout_T8_l2`): it measures disagreement between unperturbed and corrupted rollouts under the *same* action sequence, not generic rollout drift.

Predictive cost consistency (PCC), downstream readout.

$\mathrm{PCC} = \big|J(F^H(E(o), \mathbf{a}), g) - J(F^H(E(\tilde{o}), \mathbf{a}), g)\big|$ for a cost/goal readout J and goal g. PCC tests whether predictive mismatch changes the planning cost; it is a downstream readout, not the central definition.

Candidate-ranking agreement (CRA), downstream readout.

For a candidate set $\mathcal{A} = \{\mathbf{a}^1, \ldots, \mathbf{a}^K\}$ held identical across branches, the Spearman/Kendall agreement of the unperturbed and corrupted cost vectors. We treat CRA as a downstream readout, require the candidate set to be shared, and do not call it planning equivalence.

Margin-conditioned action flip (MAF), downstream readout.

$\mathrm{MAF} = \mathbf{1}\big[u^*_{\text{clean}} \neq u^*_{\text{noise}},\ C_{(2)} - C_{(1)} > \delta\big]$: a flip of the selected action counts only when the unperturbed top-1/top-2 cost margin exceeds δ, since a flip inside the noise margin need not be a failure.

Action-relevant discriminability margin (ADM), paired probe.

Over action/transition-distinct pairs P_{disc} (differing inverse-dynamics labels, large clean transition magnitude, contact/keyframe transitions, or large cost-to-go change), $\mathrm{ADM} = \mathrm{median}_{(i,j)\in P_{\text{disc}}}\, d(\Psi_i, \Psi_j)$. A consistency method should reduce ACPC_H without materially reducing ADM.

Selective predictive robustness ratio (SPRR), summary probe.

A single summary of action-distinct discriminability relative to clean/corrupted predictive inconsistency,

$$\mathrm{SPRR} = \frac{\mathrm{median}_{(i,j)\in P_{\text{disc}}}\, d(\Psi_i, \Psi_j)}{\mathrm{median}_{P_{\text{pert}}}\, \mathrm{ACPC}_H + \epsilon}.$$

We use it descriptively and do not claim it predicts success.

These diagnostics localise mechanisms and define method targets; consistent with the negative results in Section 5.5, we do not claim that any of them *predict* robustness.

4 Background and Diagnostic Framework

4.1 LeWorldModel baseline

LeWM [25] is an end-to-end JEPA world model trained with two terms only:

$$\mathcal{L}_{\text{LeWM}} \;=\; \mathcal{L}_{\text{pred}} \;+\; \lambda \cdot \mathcal{L}_{\text{SIGReg}}\,, \tag{6}$$

where $\mathcal{L}_{\text{pred}}$ is the latent-space mean-squared error (MSE) between predicted and target representations. SIGReg projects each latent onto M unit-norm random directions, computes the Epps–Pulley empirical-characteristic-function distance [9] between each projection and $\mathcal{N}(0, 1)$, and aggregates with Gauss-window weights, preventing collapse without explicit BatchNorm. (The Cramér–Wold theorem motivates the construction: equality of high-dimensional distributions reduces to equality of all one-dimensional projections of their characteristic functions.) Inference uses the Cross-Entropy Method (CEM) [22] for latent-space model predictive control (MPC) [10].

4.2 Input-side noise augmentation

We add per-frame Gaussian noise to the LeWM input pipeline via `utils.py::AddNormalizedGaussianNoise` (Section A). Each frame is independently noised: Bernoulli(p) decides whether the frame is corrupted, and if so the standard deviation is drawn from Uniform$(0, \mathrm{std_max})$. We fix $p = 1.0$ and sweep `std_max` $\in \{0.01, \ldots, 0.08\}$ (eight levels). Evaluation uses two noised conditions: pixels+goal 0.05 and pixels+goal 0.08 (Gaussian noise of the indicated std applied to both pixels and the goal image).

4.3 Five-layer diagnostic framework

The framework decomposes the JEPA world-model control forward pass — pixels $\rightarrow$ encoder f $\rightarrow$ latent z $\rightarrow$ predictor g $\rightarrow$ cost surface $\rightarrow$ planned actions (optimised with CEM) — into five sequential stages and instruments each transition with one or more metrics. Layers 1–2 characterise the encoder output; Layer 3 measures how the predictor amplifies an encoder shift; Layer 4 injects noise directly into the latent to isolate predictor and cost contributions; Layer 5 quantifies planning-relevant information retained in the latent. Most individual metrics already have a literature home (cited inline below). Our additions are scale-normalised ratios that compare perturbation effects with the unperturbed local nearest-neighbour scale. We use nearest-neighbour distance as a local latent scale primitive in the spirit of k-nearest-neighbour (kNN) out-of-distribution detection [33], but the ratios themselves are introduced here.

Relation to action-conditioned predictive consistency. The five layers below decompose a single training-time pressure (input-side noise augmentation) into probes that can separately expose nuisance contraction and loss of action-relevant discriminability. We interpret them through the predictive-consistency lens of Section 3. Layers 1–2 (encoder shift, and Centered Kernel Alignment (CKA) between unperturbed and corrupted latents) measure the *encoder perturbation difference* (EPD): whether a perturbation enters the latent at all. We treat EPD as *descriptive only*, because encoder-level closeness is neither necessary nor sufficient for control robustness (Section 3.1). Layer 5 (transition-resolution ratios R_d and the inverse-dynamics linear-probe R^2) measures action-relevant resolution — the discriminability side of Equation (5): nearby but action-distinct states must stay separable. Layer 3 (predictor sensitivity) is the layer closest to predictive consistency: its single-step fragility ratio and multi-step rollout drift are the *unconditioned* precursors of ACPC-1 and ACPC-H (Section 3.2), differing in that the ACPC quantities compare *paired* unperturbed/corrupted predictions under the *same* action sequence rather than a single rollout. Layer 4 (latent-noise response) quantifies how the cost surface reacts to direct latent perturbation. Stronger nuisance contraction never means collapsing all visually similar states: it should contract variants of the same task-relevant state while states that differ in transition, cost, or optimal action remain resolvable. Figure 1 illustrates this boundary, and Table 9 (Section 6.2) summarises where each of our four tasks sits.

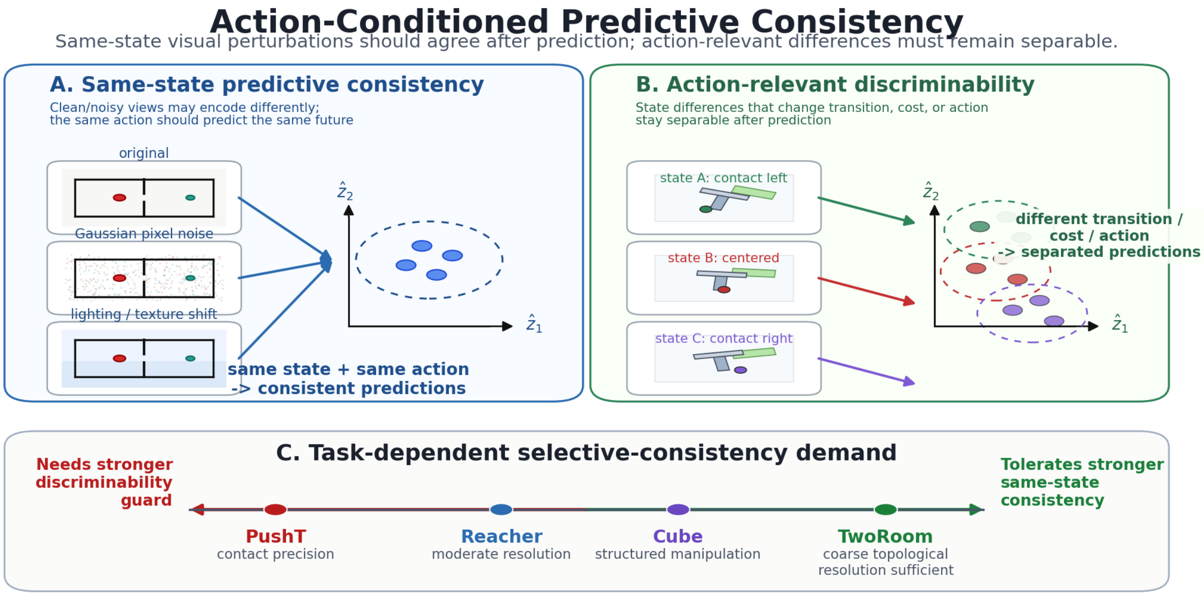


Figure 1: Conceptual reading of the selective-consistency tension. Same-state nuisance variants should induce consistent action-conditioned predictions; states that differ in transition, cost, or optimal action must remain resolvable. The bottom axis is a qualitative task read backed by the sweep and diagnostic evidence; it should not be read as "TwoRoom needs no resolution" or "PushT should preserve all pixels". PLDM is used as a second-family replication of the task-level signature, while the exact diagnostic mechanism remains method-specific. Section 3 formalises the boundary as action-conditioned predictive consistency: contract nuisance perturbations only after the action-conditioned predictor, while keeping action-distinct transitions resolvable.

Minimal notation. Let $z_i = f(x_i)$ and $\tilde{z}_i = f(\tilde{x}_i)$ be unperturbed and noised latents for the same token. The distance, local nearest-neighbour (NN) scale, and three introduced ratios are defined together as

$$\begin{aligned} d_{\cos}(a,b) &= 1 - \frac{a^\top b}{\|a\|\,\|b\|}, \\ d_i^{\mathrm{NN}} &= \min_{j\neq i} d_{\cos}(z_i, z_j), \\ r_i^{\mathrm{enc}} &= \frac{d_{\cos}(z_i,\tilde{z}_i)}{d_i^{\mathrm{NN}}+\epsilon}, \\ r_i^{\mathrm{pred}} &= \frac{d_{\cos}\big(g(z)_i, g(\tilde{z})_i\big)}{d_i^{\mathrm{NN}}+\epsilon}, \\ R_d &= \frac{\mathrm{median}_t\ d(z_t, z_{t+1})}{\mathrm{median}_{(t,t')\in\mathcal{F}}\ d(z_t, z_{t'})+\epsilon}. \end{aligned} \tag{7}$$

Here r_i^{enc} is the encoder-shift ratio, r_i^{pred} is the fragility ratio, and R_d is the transition-resolution ratio for either cosine or L2 distance d. $\mathcal{F}$ denotes random far pairs from different temporal locations, and $\epsilon = 10^{-8}$ is a numerical-stability constant. Released JSON artifacts keep the implementation keys (`noise_to_nn_cos_ratio`, `predictor_target_to_nn_cos_ratio_at_max_std`, and `transition_resolution_ratio_{cos,l2}`), but the main text uses the shorter prose names above.

Layer 1 — Encoder shift. The magnitude and direction of latent displacement under input noise. Metrics: raw angular/L2 shift, angular gain per unit noise, and the encoder-shift ratio in Equation (7).

Layer 2 — Encoder geometry. The global structure of the encoded latent space. Metrics: local neighbourhood scale, effective rank [29], and Centered Kernel Alignment (CKA) [21] between unperturbed-input and noisy-input latents at the diagnostic's maximum injection std.

Layer 3 — Predictor sensitivity. How the predictor amplifies the encoder shift. Metrics: the *fragility ratio* in Equation (7) and multi-step rollout L2 drift at horizon T.

Layer 4 — Latent-noise response. Noise injected directly into the latent z (bypassing the encoder) isolates predictor and cost contributions. Metrics: the slope of the planner's cost surface under latent perturbations and the latent robust radius, defined as the largest perturbation that keeps the cost ranking stable.

Layer 5 — Task resolution. The control-relevant information retained in the latent. Metrics: the L2/cosine transition-resolution ratio R_d in Equation (7) and the inverse-dynamics linear-probe R^2, a controllability proxy in the spirit of linear-probe analysis [1].

Reading guidance. Layers 1, 2, 4, and 5 supply descriptive evidence used throughout this paper (the per-task radar in Figure 3 and the full base profile in Table 10). Layer 3 is the only layer whose two full-coverage metrics — the fragility ratio and the multi-step rollout drift — are used as cross-checkpoint predictors in the partial-correlation analysis of Section 5.5; the other layers are kept descriptive because either their probe definition is not single-valued per checkpoint (Layers 1, 4) or their metric is saturated in some tasks (Layers 2, 5).

4.4 Cross-checkpoint validation protocol

To rule out spurious correlations we adopt two complementary analyses on the same LeWM PushT sweep:

LeWM $n = 9$ sweep with partial correlation. All 9 LeWM checkpoints per task (base + `std_max` $\in \{0.01, \ldots, 0.08\}$). Compute Spearman ρ and *partial* Spearman conditioned on `std_max`. The partialling step matters: many diagnostics correlate with control performance only because both quantities co-vary with the training noise level along the sweep. The relevant question is whether a diagnostic retains a residual checkpoint-quality signal after removing the monotone sweep trend associated with `std_max`.

We report both the raw Spearman and the partial-on-`std_max` quantities. The raw ρ tells you "which checkpoint over the entire sweep behaves better"; the partial ρ tells you whether a diagnostic retains a residual checkpoint-quality signal after removing the monotone trend associated with `std_max`. The two questions are distinct, and we will see in Section 5.5 that they have markedly different answers for the same metric. Each reported partial Spearman value is paired with a 95% percentile bootstrap confidence interval ($B = 1000$ with-replacement resamples of the checkpoint rows within scope; seed = 42). The bootstrap output is released as `assets/paper1_data/partial_corr_bootstrap_20260523.json`, and the regeneration script is `tools/build_partial_corr_bootstrap.py`. PLDM provides a first second-family replication in Section F; broader cross-architecture variants such as frozen/pretrained visual features remain follow-up work.

5 Experiments

5.1 Setup

Tasks. PushT (2D pushing), TwoRoom (2D navigation), Reacher (2D arm), Cube (3D manipulation). This is a deliberately heterogeneous stress panel rather than a benchmark claim: TwoRoom supplies visually redundant discrete navigation; PushT supplies contact-heavy planar manipulation where small visual differences change the next useful action; Reacher supplies low-dimensional continuous control; Cube supplies structured 3D manipulation with predictable visual–action coupling. The panel is therefore chosen to span different nuisance/discriminability regimes under a shared visual-corruption protocol, not to estimate an average over all control tasks.

Baselines. LeWM-base (no noise) and LeWM+noise (8-level sweep).

Checkpoints. The 36 checkpoints in this paper correspond to one trained model per $(\text{task}, \text{configuration})$: one no-noise baseline plus eight `std_max` values for each of the four tasks.

Evaluation seeds. Each checkpoint is evaluated with 3 evaluation seeds (42 / 43 / 44), with 100 trajectories per seed.

Evaluation protocol. All success rates in this paper — unperturbed and noised, across all 36 checkpoints (4 tasks $\times$ {base, `std_max` 0.01–0.08}) — are computed under a single protocol: $n = 3$ seeds (42/43/44), `num_eval = 100` trajectories per seed (300 trajectories per condition per checkpoint). We use *clean* only as the conventional artifact/condition name for unperturbed original evaluation images. Every cell of Tables 1, 2, 3 is mean $\pm$ across-seed population std over $n = 3$ and is sourced from the released aggregate, `assets/paper1_data/canonical_evals_20260517.json`; raw per-seed files live alongside each checkpoint as `<ckpt>/eval_results/<cond>_seed{42,43,44}_metrics.txt`. Success-rate tables report across-seed population std, while correlation intervals use checkpoint-level bootstrap CIs; these quantify different uncertainty sources. The diagnostic source-of-truth for Figure 4, Table 5, Table 6, and Table 7 is `assets/paper1_data/canonical_diagnostics_20260517.json`.

Hardware. Single NVIDIA H800 (80 GB).

Figures. Five main figures (Figures 1 to 5) are produced by `tools/paper1_figs.py`.

5.2 JEPA control fragility under visual corruptions

Table 1 reports LeWM-base success rates under unperturbed and noised evaluation (mean $\pm$ std across 3 seeds $\times$ 100 evaluations).

Table 1: LeWM-base under test-time visual corruptions (mean $\pm$ std; 3 seeds $\times$ 100 evaluations).

Task	unperturbed	px+goal 0.05	px+goal 0.08	unperturbed $\rightarrow$ 0.08 drop
TwoRoom	94.00 ± 3.56	61.33 ± 5.31	50.00 ± 1.41	-44.00
PushT	86.33 ± 2.36	12.00 ± 4.55	4.67 ± 2.05	-81.67
Reacher	58.67 ± 1.25	27.00 ± 5.10	15.00 ± 2.16	-43.67
Cube	66.67 ± 2.62	53.33 ± 3.30	46.33 ± 3.68	-20.33

LeWM-base is strong on unperturbed images (especially TwoRoom and PushT), but visual std = 0.05 applied to pixels and goal jointly already produces large drops on all tasks: PushT loses 74+ pts (down to near-random 4.67% at std = 0.08), TwoRoom 30+ pts, Reacher 30+ pts, Cube $\sim$ 13 pts (at std = 0.05). This is a large closed-loop effect rather than a marginal representation-space shift. The drop pattern across tasks is informative: Cube degrades least (-20.33 pt at std = 0.08) — structured manipulation is less sensitive in this protocol — while PushT degrades most catastrophically (-81.67 pt), confirming that contact-heavy continuous control is most sensitive to visual precision.

The same direction of failure is visible on PLDM but with task-dependent strength. PLDM trained without noise augmentation drops sharply on PushT (75.33% $\pm$ 3.68 unperturbed to 10.00% $\pm$ 2.16 at px+goal 0.08, -65.33 pt) and TwoRoom (96.67% $\pm$ 1.70 to 51.67% $\pm$ 2.05, -45.00 pt), while Reacher and Cube have much smaller no-noise-training gaps (-3.33 and -4.33 pt; Table 14). The noise-training signatures still carry over: TwoRoom recovers into a high-noise plateau, PushT has a large `std_max` ≈ 0.03 recovery, and Cube responds weakly (Section F).

5.3 Noise augmentation closes the gap — at the cost of task-specific tuning

Tables 2 and 3 report the complete 8-level sweep across tasks. All cells are success rate $\pm$ across-seed std (in pts), aggregated under the unified $n = 3$ seeds (42/43/44) $\times$ 100 trajectories protocol — 300 trajectories per cell.

Table 2: LeWM+noise sweep, unperturbed (clean condition-name) success rate (%).

`std_max`	TwoRoom	PushT	Reacher	Cube
0 (base)	94.00 $\pm$ 3.56	86.33 $\pm$ 2.36	58.67 $\pm$ 1.25	66.67 $\pm$ 2.62
0.01	93.67 $\pm$ 3.30	88.00 $\pm$ 3.74	61.67 $\pm$ 2.49	69.33 $\pm$ 0.47
0.02	95.00 $\pm$ 2.83	88.33 $\pm$ 2.62	85.67 $\pm$ 2.49	60.00 $\pm$ 1.63
0.03	96.33 $\pm$ 3.30	89.67 $\pm$ 1.70	78.67 $\pm$ 1.25	65.00 $\pm$ 1.63
0.04	96.33 $\pm$ 2.05	89.33 $\pm$ 2.05	84.00 $\pm$ 2.94	69.00 $\pm$ 3.74
0.05	96.00 $\pm$ 2.83	80.67 $\pm$ 4.78	70.00 $\pm$ 2.16	59.33 $\pm$ 0.94
0.06	96.67 $\pm$ 2.05	89.33 $\pm$ 2.05	86.00 $\pm$ 2.94	66.67 $\pm$ 2.05
0.07	96.00 $\pm$ 1.63	85.67 $\pm$ 3.09	83.67 $\pm$ 3.30	67.67 $\pm$ 0.94
0.08	98.33 $\pm$ 0.47	88.33 $\pm$ 2.87	84.00 $\pm$ 0.82	62.33 $\pm$ 1.25

Table 3: LeWM+noise sweep, px+goal 0.08 success rate (%).

`std_max`	TwoRoom	PushT	Reacher	Cube
0 (base)	50.00 $\pm$ 1.41	4.67 $\pm$ 2.05	15.00 $\pm$ 2.16	46.33 $\pm$ 3.68
0.01	87.67 $\pm$ 1.89	43.33 $\pm$ 3.09	46.00 $\pm$ 1.63	51.33 $\pm$ 5.79
0.02	93.33 $\pm$ 0.94	71.33 $\pm$ 3.68	85.67 $\pm$ 1.70	60.67 $\pm$ 0.47
0.03	94.67 $\pm$ 2.87	83.00 $\pm$ 3.74	73.67 $\pm$ 0.47	67.33 $\pm$ 1.89
0.04	95.00 $\pm$ 2.45	81.33 $\pm$ 2.87	80.00 $\pm$ 1.41	67.00 $\pm$ 3.56
0.05	95.67 $\pm$ 2.36	75.00 $\pm$ 6.48	68.00 $\pm$ 3.56	59.67 $\pm$ 2.05
0.06	96.67 $\pm$ 2.49	87.00 $\pm$ 3.74	84.67 $\pm$ 4.03	65.00 $\pm$ 2.94
0.07	96.33 $\pm$ 2.05	82.33 $\pm$ 4.64	81.33 $\pm$ 1.25	68.00 $\pm$ 1.41
0.08	98.67 $\pm$ 0.94	85.33 $\pm$ 2.62	83.00 $\pm$ 4.32	60.33 $\pm$ 0.94

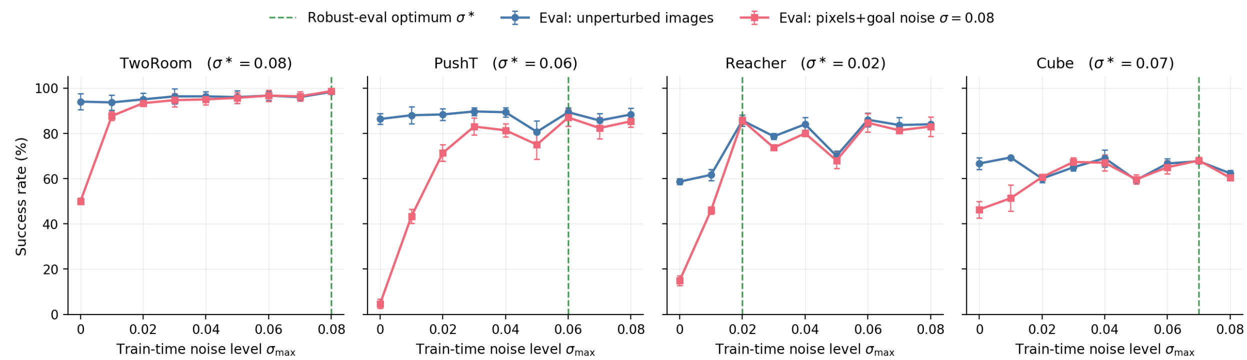


Figure 2: Noise-training sweep across all four tasks. Blue: success rate under unperturbed evaluation. Red: success rate under pixels+goal Gaussian noise $\sigma = 0.08$. Green dashed: each task's robust-eval optimum σ^* (the train-time `std_max` that maximises corrupted-eval success). The optimum σ^* varies across tasks, consistent with a coarse global scalar pressure with broad, task-dependent plateaus rather than a single jointly optimal level.

Three observations follow.

(1) The optimal `std_max` varies across tasks; unperturbed and robust optima can dissociate within a task. TwoRoom peaks globally at `std_max` = 0.08 (98.33 / 98.67); unperturbed success rises monotonically with noise. PushT peaks on unperturbed evaluation at `std_max` = 0.03 but on robustness at `std_max` = 0.06 (87.00 vs. 0.02's 71.33; +15.67 pt). Reacher lies on a 0.02–0.06 plateau: the unperturbed point-best is at `std_max` = 0.06 (86.00), while px+g 0.08 point-best is at `std_max` = 0.02 (85.67). At `std_max` = 0.01 unperturbed success is 61.67 vs base 58.67 — within across-seed std ($\sim$ 2.5 pts), so the data support *"low noise is statistically equivalent to base"*, not "low noise hurts". Cube responds least: unperturbed success is non-monotonic, and px+g 0.08 improves only in the 0.03–0.07 range.

(2) Per-task tuning is necessary, not optional. Optimal `std_max` varies substantially: TwoRoom unperturbed/corrupted point-best 0.08, PushT unperturbed point-best 0.03 / px+g 0.08 point-best 0.06, Reacher unperturbed point-best 0.06 / px+g 0.08 point-best 0.02, Cube px+g 0.08 point-best 0.07 with a shallow unperturbed plateau around 0.01 / 0.04 / 0.07. Global input-side noise is a coarse same-state consistency pressure, but closing the corruption gap requires per-task tuning cost.

(3) Tasks form a clear sensitivity ordering at the extremes. PushT is clearly most sensitive (−81.67 base drop), Cube least sensitive (−20.33), while TwoRoom (−44.00) and Reacher (−43.67) are effectively tied around a 44-pt drop. Recovery does not scale with sensitivity: TwoRoom recovers most fully (+48.67 pt at `std_max` = 0.08), Cube least (+21.67 pt). Input-side global noise is most effective on visually redundant tasks and gives diminishing returns on structured manipulation.

The behavioural evidence for this selective-consistency tension is summarised by the sweep tables and Figure 2; the representation-level evidence appears in Table 4.

5.4 Diagnostic analysis: why global noise is only a coarse pressure

Where the per-task sweep evidence in Section 5.3 is the behavioural face of selective consistency, the diagnostics in this subsection are its representational face: which latent properties move under noise training, and whether the movement is bounded by each task's discriminability requirements. Table 4 compares core diagnostic metrics on LeWM-base versus the per-task representative noise-trained diagnostic checkpoint.

Table 4: Representation diagnostics: LeWM-base vs. a representative noise-trained diagnostic checkpoint per task. The σ pick per task (0.08 / 0.02 / 0.06 / 0.01) is the ckpt on which the diagnostic suite was originally executed. Under the unified 3-seed $\times$ 100 eval protocol the PushT unperturbed point-best shifts to `std_max` $= 0.03$ (within ± 2 pt of `std_max` $= 0.02$); we keep the diagnostics on the `std_max` $= 0.02$ checkpoint because the compression-vs.-resolution pattern this table illustrates is robust within this neighbourhood.

Metric	TwoRoom base	TwoRoom noise (0.08)	PushT base	PushT noise (0.02)	Reacher base	Reacher noise (0.06)	Cube base	Cube noise (0.01)
`clean_nn_cos_dist_median`	0.0449	0.0281	0.2360	0.1051	0.0633	0.0676	0.1856	0.1879
`clean_effective_rank`	47.60	33.59	76.42	42.85	61.04	65.92	73.25	71.83
`transition_resolution_ratio_l2`	0.7216	0.6055	0.3015	0.2800	0.3704	0.3791	0.4847	0.4629
`transition_resolution_ratio_cos`	0.5538	0.3780	0.0868	0.0800	0.1351	0.1399	0.2347	0.2168
`id_probe_r2`	0.2889	−0.0573	0.7739	0.7500	0.1621	0.1729	0.6657	0.6720
`action_mean_pred_shift_norm`	0.5329	0.4482	0.1283	0.1200	0.2518	0.2585	0.2364	0.2320
`predictor_rollout_T8_l2`	18.62	17.90	18.65	16.50	15.17	0.44	20.20	19.25

Notes. (i) Reacher and Cube representative diagnostics are taken from the corresponding checkpoint's per-tool JSON outputs under the diagnostics directory (max-std $= 0.1$, history-only noise); (ii) the Reacher representative rollout drift of 0.44 (a $35\times$ reduction from base 15.17) is not a recording error: Reacher's representative checkpoint trains under `std_max` $= 0.06$, which is enough to collapse the multi-step predictor drift while keeping single-step task-resolution metrics flat — precisely the dissociation that motivates the analysis in Section 5.5. Cube's representative drift ($19.25 \approx$ base 20.20) is the opposite extreme.

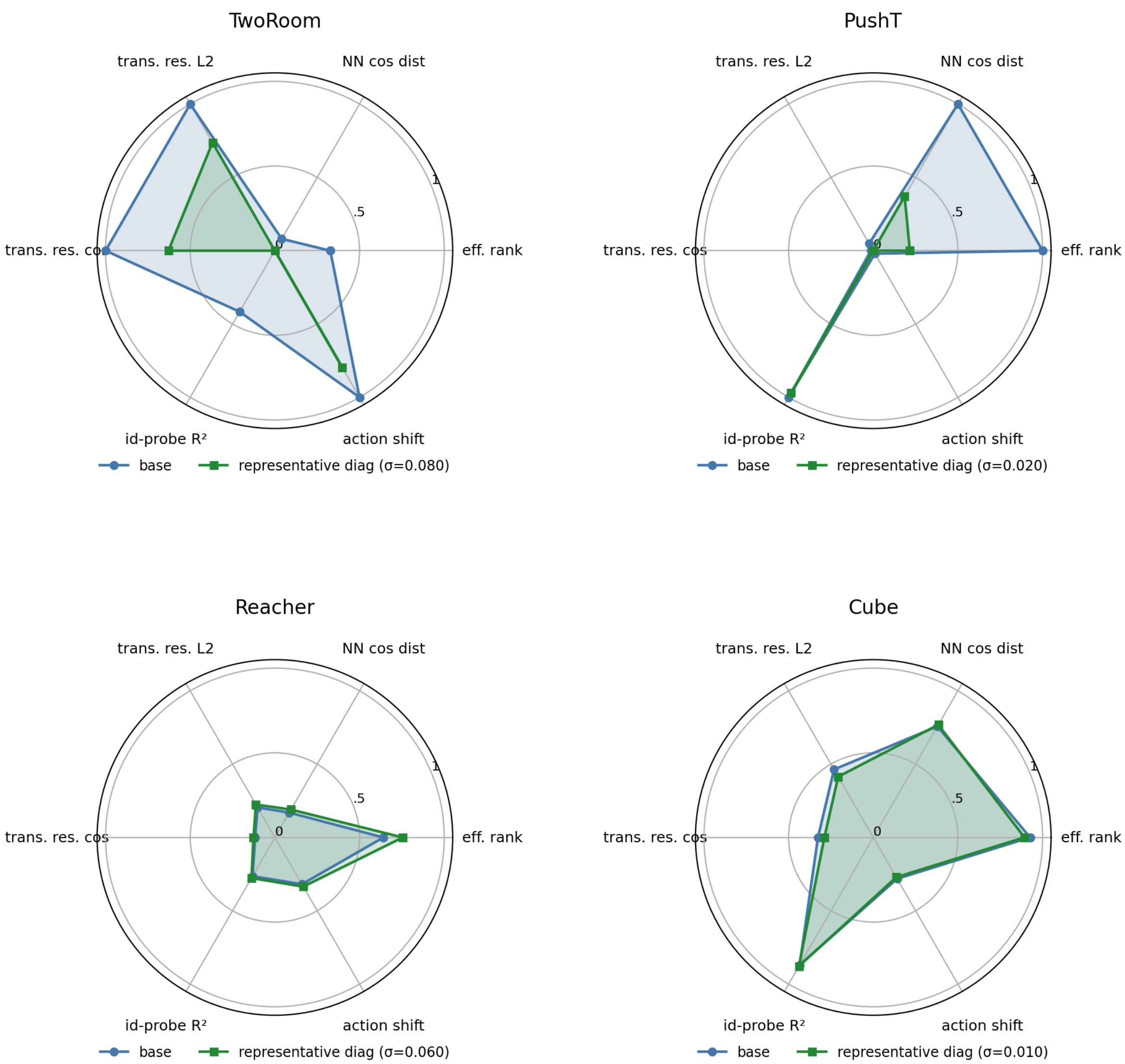


Figure 3: Per-task diagnostic radar: base vs representative noise-trained diagnostic checkpoint on 6 metrics, min-max normalised across all 4 tasks.

Mechanistic reading.

- **TwoRoom.** Low-dimensional, discrete, visually redundant. Compressing the representation (effective rank $47.6 \rightarrow 33.6$) is acceptable and even beneficial: a more compact latent space is easier to plan in.
- **PushT.** Continuous contact requires fine-grained pose resolution. Even at the representative diagnostic checkpoint (`std_max` $= 0.02$), the L2 transition-resolution metric already trends slightly downward. At heavier noise (e.g. 0.06) it drops further, erasing contact-transition keyframes.
- **Reacher.** Low-dimensional continuous reaching does not show a resolution collapse in Table 4: effective rank, transition resolution, and the controllability probe remain flat or slightly improve. The notable change is the large reduction in multi-step predictor drift ($15.17 \rightarrow 0.44$), matching the later finding that Reacher's residual corruption-drop signal lives in the rollout metric rather than in the single-step fragility ratio.
- **Cube.** Cube is moderately resolution-demanding but less fragile than PushT. The representative

checkpoint leaves rank, transition resolution, controllability probe, and rollout drift almost unchanged, consistent with Cube's smaller visual-corruption cliff and the moderate residual signal of multi-step drift in Table 6.

- **Predictor-rollout caveat.** A drop in multi-step predictor rollout drift is not unambiguously good news. It can also mean the latent has become more *predictable* without being more *controllable*: predictor stability bought by sacrificing resolution.

5.5 Cross-checkpoint correlation analysis

Table 5 reports task-specific cross-checkpoint correlations on the LeWM $n = 9$ sweep using the unified 3-seed $\times$ 100 evaluation.

We analyse two single-value-per-checkpoint diagnostic metrics that have full coverage on all 9 checkpoints per task: the fragility ratio, defined in Section 4.3 as the single-step predictor target shift normalised by the nearest-neighbour cosine distance at the diagnostic's max-std injection, and the corresponding multi-step rollout L2 drift. Both are released in `assets/paper1_data/canonical_diagnostics_20260517.json`, derived from each checkpoint's predictor-sensitivity diagnostic JSON, and are pure checkpoint-level quantities (independent of the evaluation protocol).

Table 5: LeWM $n = 9$ sweep: per-task Pearson r / Spearman ρ vs corruption drop (unperturbed − px+g 0.08). Eval values come from the unified 3-seed $\times$ 100 protocol.

Metric	TwoRoom (r/ρ)	PushT (r/ρ)	Reacher (r/ρ)	Cube (r/ρ)
`predictor_target_to_nn_cos_ratio_at_max_std`	+0.96/+0.72	−0.54/−0.77	+0.97/+0.35	+0.36/+0.21
`predictor_rollout_T8_l2_at_max_std`	+0.89/+1.00	+0.99/+0.88	+0.99/+0.87	+0.92/+0.54

Reading. Unconditional correlations are large because both diagnostic metrics and the corruption drop are driven by the same upstream variable — `std_max`. The relevant test is partial correlation conditioned on `std_max` (Table 6).

Table 6: Partial Spearman ρ(metric, corruption drop | `std_max`), $n = 9$ per task.

Metric	TwoRoom	PushT	Reacher	Cube
`predictor_target_to_nn_cos_ratio_at_max_std`	— (rank ties)	+0.06	−0.12	+0.14
`predictor_rollout_T8_l2_at_max_std`	— (rank ties)	−0.03	+0.79	+0.50

The TwoRoom partial estimates are undefined because saturating success rates produce rank ties that make residualisation singular at $n = 9$. PushT, Reacher and Cube give clear readings: after removing the monotone sweep trend associated with `std_max`, the **fragility metric carries essentially no information about the size of the corruption drop** on any task; the **multi-step predictor drift** still carries signal on Reacher (+0.79) and weak signal on Cube (+0.50). The Reacher partial ρ is the only non-trivial residual correlation in the entire matrix.

Table 7: Spearman ρ (unconditional) and partial Spearman ρ conditioned on `std_max`, for the fragility ratio on the PushT $n = 9$ LeWM sweep under the unified 3-seed $\times$ 100 eval. CIs are the percentile 95% interval over $B = 1000$ bootstrap resamples (with replacement, seed = 42; see Section 4.4). The `std_max`-only top four rows are univariate sweep diagnostics with no metric/outcome pairing, so no CI is reported.

Quantity	Spearman ρ	95% bootstrap CI
ρ(`std_max`, metric)	+0.83	—
ρ(`std_max`, unperturbed)	0.00	—
ρ(`std_max`, px+goal 0.08)	+0.82	—
ρ(`std_max`, corruption drop)	−0.93	—
ρ(metric, unperturbed) unconditional	−0.33	[−0.95, +0.62]
ρ(metric, unperturbed) | `std_max` partial	−0.59	[−0.97, −0.10]
ρ(metric, px+g 0.08) unconditional	+0.55	[−0.28, +0.86]
ρ(metric, px+g 0.08) | `std_max` partial	−0.41	[−0.77, +0.00]
ρ(metric, corruption drop) unconditional	−0.77	[−0.95, −0.12]
ρ(metric, corruption drop) | `std_max` partial	+0.06	[−0.00, +0.25]

PushT $n = 9$ detailed view (fragility ratio). Interpretation:

1. **After removing the monotone trend associated with `std_max`, the metric still carries a meaningful residual checkpoint-quality signal.** Partial ρ on unperturbed evaluation is −0.59 and on px+goal 0.08 is −0.41 — both negative, both meaningful at this sample size. On the $n = 9$ PushT sweep, lower fragility ratio aligns with better unperturbed *and* better noisy success beyond what the sweep-level `std_max` trend already explains.
2. **The metric does NOT predict the unperturbed-to-corrupted gap.** The unconditional $\rho(\text{metric}, \text{drop}) = -0.77$ looks impressive, but partialling out `std_max` collapses it to +0.06 (sign-flipped, near zero; 95% bootstrap CI [−0.00, +0.25], including 0). The drop's strong correlation with the metric is a mediated effect: `std_max` drives both the metric ($\rho = +0.83$) and the drop ($\rho = -0.93$). After the `std_max` trend is removed, the metric tells you nothing new about how much the *gap* between unperturbed and noisy performance will be.
3. **Note the unconditional-vs-partial sign reversal on unperturbed evaluation.** ρ(metric, unperturbed) is only −0.33 unconditionally — unperturbed success rate is roughly flat across the PushT sweep under the 3-seed protocol (range 80.67–89.67), so `std_max` barely moves unperturbed performance. The partial correlation *strengthens* to −0.59 once the sweep-level `std_max` trend is removed, exactly because the residual signal is what the metric tracks.
4. **Practical reading.** The toolkit is useful for mechanism localisation and checkpoint selection after controlling for the sweep-level `std_max` effect. It is *not* a substitute for actually training and evaluating under corruptions when the robustness gap is the quantity of interest.

5.5.1 Unperturbed control fidelity and corruption robustness as separable signals

The partial-correlation analysis above settles the interpretation:

- The metric is a **residual checkpoint-quality signal beyond the sweep-level `std_max` effect** — a lower fragility ratio means better control on *both* unperturbed and noisy evaluation (partial $\rho = -0.59$ / -0.41 on PushT).
- The metric **does not isolate noise robustness** — its apparent correlation with the gap between unperturbed and noisy success is fully mediated by `std_max` (partial $\rho = +0.06$).

The PushT scatter (Figure 4) makes both halves visible. Panel (a) plots metric vs unperturbed success (unconditional Spearman $\rho = -0.33$; the residual trend after conditioning on `std_max` is what the partial correlation captures). Panel (b) plots metric vs corruption drop (unconditional

$\rho = -0.77$), but the colour-bar (encoding `std_max`) reveals the structure: low-`std_max` checkpoints (light blue) live at high drop, high-`std_max` checkpoints (dark blue) live at low drop, and the metric tracks `std_max` itself. After the `std_max` trend is removed, the metric does not separate small-drop from large-drop checkpoints.

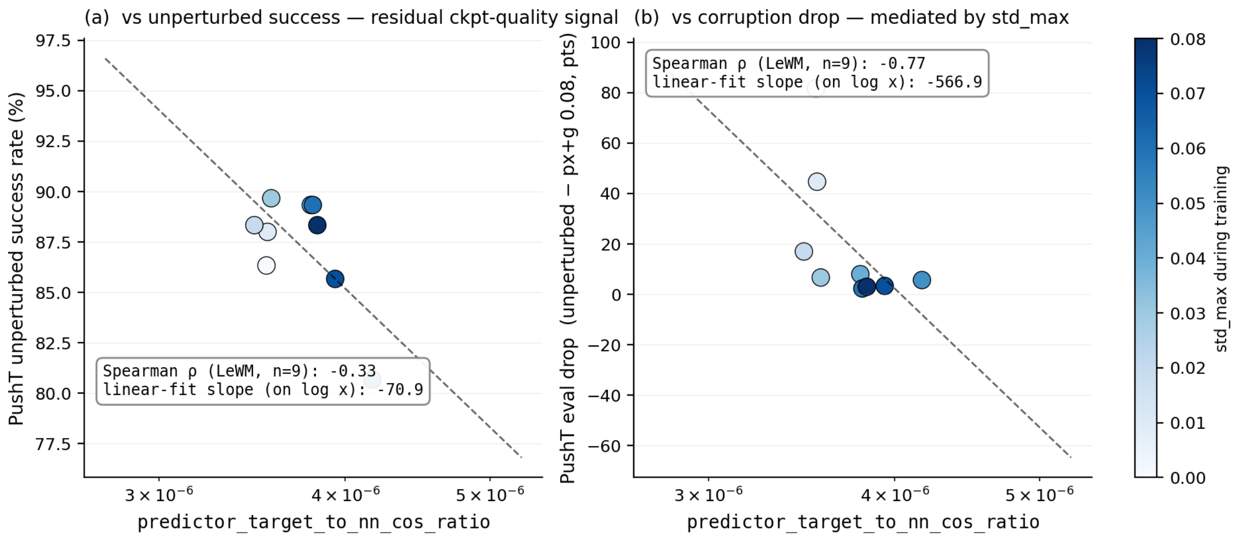


Figure 4: PushT $n = 9$ LeWM sweep: the fragility ratio is a checkpoint-quality predictor (a); the apparent corruption-drop correlation in (b) is mediated by `std_max` (encoded by the colour bar).

5.6 Mechanism attribution: where in the pipeline does noise cause failure?

Section 5.4 reports *what* is compressed and Section 5.5 reports *which diagnostics* correlate with eval across checkpoints, but neither answers **"is the failure in the encoder, the predictor, or the cost surface?"** We address this with two complementary experiments.

5.6.1 Auxiliary cost-swap sanity check: cost alone is unlikely to explain the collapse

We ran a one-off eval-only cost swap on a single TwoRoom checkpoint outside the 36-checkpoint canonical evaluation table; full details are reported in Section E. Switching the CEM cost from cosine/normalized to mse/raw changes px+goal 0.03 success only from 36.0 to 42.0, still far below a separate unperturbed reference of 69.7 (separate $n = 300$ eval, single seed; see Section E). As a sanity check, this suggests that the cost function alone is unlikely to explain the visual-corruption collapse.

5.6.2 Latent-noise probing: encoder shift is a major contributor

Directly injecting noise into the latent z (skipping the encoder) decouples encoder contributions from predictor + cost. Comparing diagnostic signals across two injection points (pixels vs. latent z, both history-only noise at max std):

- **PushT.** Multi-step input-space rollout drift has unconditional $\rho = +0.88$ vs corruption drop (Table 5), driven primarily by `std_max`; after removing the monotone `std_max` trend, the partial ρ collapses to -0.03. The single-step fragility ratio (unconditional $\rho = -0.77$; partial $+0.06$ after removing the same trend) tells the same story. Both encoder–predictor signals are mediated by training noise; once that sweep-level effect is accounted for, neither isolates the corruption drop on PushT.

- **Reacher.** Multi-step rollout drift has unconditional $\rho = +0.87$ *and* partial $\rho = +\mathbf{0.79}$ vs corruption drop (95% bootstrap CI $[+0.00, +1.00]$, borderline-significant at the 95% level) — the only non-trivial residual correlation in the matrix. On Reacher, multi-step predictor drift carries genuine corruption-drop information beyond `std_max`.
- **Cube.** Multi-step rollout drift has unconditional $\rho = +0.54$ and partial $\rho = +0.50$ (95% CI $[-0.30, +1.00]$, marginal) — a moderate residual signal. Encoder–predictor drift partially explains Cube's small but non-zero corruption sensitivity.
- **TwoRoom.** Partial correlations are undefined because unperturbed success / drop are saturated across the sweep (rank ties). Unconditional ρ still indicates strong encoder–predictor sensitivity to `std_max`.

Table 8 summarises the three-layer attribution.

Table 8: Three-layer attribution by task (LeWM $n = 9$ sweep, unified 3-seed $\times$ 100 eval).

Task	Strongest signal in these probes	Residual after removing the `std_max` trend
TwoRoom	encoder-shift signal (rank-saturated)	n/a
PushT	encoder + single-step predictor (both mediated by `std_max`)	no residual metric isolates corruption drop
Reacher	encoder + multi-step rollout	multi-step drift residual signal (partial $\rho = +0.79$)
Cube	encoder	moderate residual on multi-step drift (partial $\rho = +0.50$)

Across these probes, the strongest common signal is **encoder shift transduced by the predictor**. The auxiliary cost-swap sanity check in Section 5.6.1 suggests that the cost function alone is unlikely to explain the collapse, but we do not treat that single-checkpoint ablation as a task-wide quantitative attribution. A stronger causal claim would require multi-task latent-injection or cost-ranking ablations. The present evidence is sufficient for mechanism localisation: once the encoder's latent-neighbourhood structure is corrupted beyond the NN distance scale, downstream predictor and planner can operate on the wrong neighbourhood.

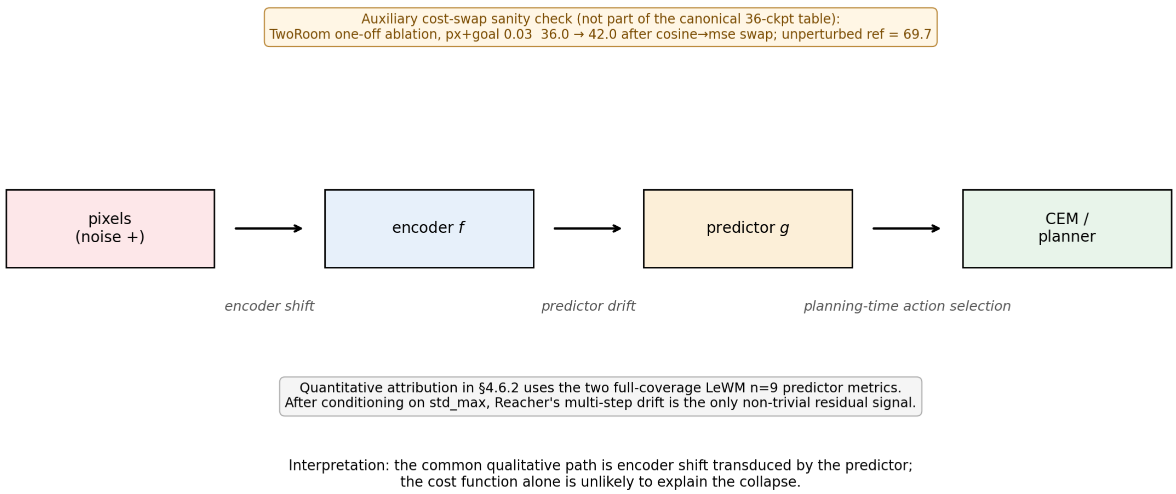


Figure 5: Mechanism schematic. Pixels perturb the encoder, the predictor transduces that shift into planning-relevant latent drift, and CEM acts on the resulting latent geometry. Quantitative attribution in §5.6.2 uses the two full-coverage LeWM $n = 9$ predictor metrics; after conditioning on `std_max`, Reacher's multi-step drift is the only non-trivial residual signal.

6 Discussion

6.1 From latent invariance to predictive consistency

The data challenge an implicit community assumption: latent prediction alone does not confer robustness to visual noise. But the corrective is *not* to demand that corrupted and unperturbed images map to the same latent. Our results are better read through the predictive-consistency lens of Section 3: encoder-level latent closeness is neither necessary nor sufficient (Section 3.1), so the target is not $z \approx \tilde{z}$ but agreement of the action-conditioned predictions, $F^k(z, \mathbf{a}) \approx F^k(\tilde{z}, \mathbf{a})$, in task-relevant coordinates. The invariance a JEPA encoder acquires is *in-distribution* invariance; when test-time inputs carry pixel noise unseen during training, the latent neighbourhood structure breaks down, the predictor rolls out the wrong future, and the planner fails. What matters for control is whether same-state perturbations stay predictive-consistent while action-sensitive state differences stay discriminable.

This should be read alongside VJEPA [18], which reports that JEPA-family models keep signal-recovery $R^2 > 0.84$ under a synthetic Noisy-TV distractor. That positive result is compatible with ours because the evaluation modalities differ: signal-recovery probes only need the latent to preserve recoverable state information, whereas closed-loop control demands a representation *and* predictor that support precise action optimisation — exactly the action-conditioned predictions on which we place the consistency requirement.

6.2 Task-dependent manifestation and scope

The selective-consistency demand — contract nuisance perturbations of the same state, preserve action-relevant transitions — has the following four-task signature in this paper. We use two qualitative task-property labels below: "nuisance-contraction demand" is the room to contract irrelevant visual variation, and "control-discriminability demand" is the action-relevant information that must survive.

- **TwoRoom.** Background wall colour / texture is pure visual redundancy; discarding it does not impair navigation. This does not mean that all positions can collapse: room, doorway, and goal-topology distinctions must remain separable.
- **PushT.** The pixel changes during T-block / arm contact carry critical force and pose information; discarding them blinds the planner.
- **Reacher.** Low-dimensional continuous control; visual encoding of joint angle requires moderate resolution.
- **Cube.** Object pose and grasp-point spatial relations need moderate resolution, but Cube itself is largely insensitive to global noise (action sequence is structured; visual–action coupling is predictable).

Task-specificity is why input-side noise behaves as a coarse global pressure rather than a single jointly optimal level. Table 9 consolidates the behavioural (Section 5.3) and representational (Section 5.4) evidence into a single per-task read.

Table 9: Where each LeWM task sits under the selective-consistency tension. The two "demand" columns are qualitative task-property reads, not independent measurements; they index the underlying behavioural and representational evidence in Sections 5.3 and 5.4 and the operational definition in Section 4.3. Sweep signatures come from Tables 2 and 3; diagnostic readings from Table 4.

Task	Nuisance-contraction demand	Control-discriminability demand	Observed sweep signature	Diagnostic reading at representative ckpt
TwoRoom	high (visual redundancy)	low–moderate (discrete navigation)	heavy-noise plateau ($\sigma^* = 0.08$)	compact latent acceptable; rank 47.6 → 33.6 without controllability loss
PushT	moderate	high (contact precision)	dissociated optima ($\sigma^*_{\text{unp}} = 0.03$, $\sigma^*_{\text{corr}} = 0.06$)	compression risks resolution; rank 76.4 → 42.9 with transition-resolution and inverse-dynamics-probe downward movement
Reacher	moderate	moderate	broad plateau ($\sigma^* \in 0.02$–0.06)	rollout drift carries the residual signal more than rank or single-step resolution
Cube	low–moderate	moderate	weak response ($\sigma^* = 0.07$)	structured action coupling buffers noise; small representation movement

Scope. Whether this task-dependent pattern generalises to other latent world-model architectures is an open question we do not fully address. Reconstruction-based world models such as DreamerV3 [15] explicitly model observations, while decoder-free latent MPC systems such as TD-MPC2 [16] may exhibit different compression dynamics. ViGMO [26] reports related task-specific sensitivity to sensor noise (Gaussian noise and blur) on DeepMind Control (DMC). Exponential-moving-average (EMA) target JEPA variants (I-JEPA / V-JEPA lineage) and variational JEPA may exhibit different dynamics. Our mechanism claim is LeWM-centred, with PLDM used as a second-family replication for the task-level fragility/recovery signature and the PushT partial-correlation null; universalising the compression chain exceeds the present evidence.

6.3 When the diagnostic toolkit works — and when it does not

Three scope conditions should be stated up-front so practitioners know when to use the toolkit and when to fall back to direct evaluation.

Scope 1: the toolkit ranks residual checkpoint quality, not corruption-specific robustness. Section 5.5.1 shows that on PushT, after conditioning on `std_max`, the strongest cross-checkpoint diagnostic (the fragility ratio) has partial Spearman $\rho = -0.59$ with unperturbed success and $\rho = -0.41$ with px+goal 0.08 success. It does *not* isolate corruption-specific robustness: the partial correlation with the unperturbed-to-corrupted drop is only $+0.06$. Use the toolkit to pick the strongest checkpoint after removing the sweep-level `std_max` trend; do not use it as a substitute for an actual corruption eval if your downstream question is robustness.

Scope 2: tasks with weak per-state controllability variance fall outside the toolkit's strict regime. Reacher and TwoRoom do not produce a diagnostic-vs-eval Spearman ρ that survives the partial-correlation criterion. Both tasks lack enough residual spread after accounting for `std_max` to distinguish "good" from "bad" checkpoints via a label-free metric. The toolkit can describe what these models have compressed (Table 4) but cannot predict relative checkpoint quality.

Scope 3: cross-checkpoint diagnostics cannot recover what training did not provide. The largest determinant of the corruption drop is whether the model saw noise during training — not which fragility ratio it happens to score on. This is the structural reason $\rho(\text{metric}, \text{drop})$ is weak even when $\rho(\text{metric}, \text{unperturbed})$ is strong: noise vs no-noise training puts each checkpoint on a completely different (unperturbed, corrupted) curve, and no static cross-checkpoint diagnostic can substitute for that training-time choice.

The toolkit therefore has a precise scope: it is an *unperturbed-evaluation auxiliary* that lets you select among checkpoints on tasks with strong per-state controllability variance (PushT, Cube), assuming the training protocol is already fixed. It is useful for mechanism localisation and checkpoint selection within a fixed protocol, but it is not a robustness oracle.

6.4 Practical guidance: how to choose `std_max` on a new task

Our sweep suggests a simple operational recipe:

1. Inspect the no-noise baseline's fragility ratio as a screening diagnostic, not as a robustness oracle. Very small values (PushT / Cube regime) indicate that local encoder–predictor shift is small relative to the unperturbed nearest-neighbour scale, but they do not by themselves rank corruption sensitivity.
2. Check effective rank, transition-resolution ratio, and task semantics together. PushT combines high rank and high controllability ($\texttt{id\_probe\_r}^2 = 0.7739$), so noise should be swept with unperturbed performance as a guardrail. TwoRoom is visually redundant and retains high L2 transition separability ($R_{\mathrm{L2}} = 0.7216$), so a wider sweep up to 0.08+ is reasonable.
3. Use *two endpoints* in evaluation (unperturbed and max-noise). Checking only one misses one of the optima: PushT's unperturbed optimum at 0.03 vs robustness optimum at 0.06 is the clearest example.
4. Under compute budget, start with a coarse 4-level sweep ($\{0.01, 0.03, 0.05, 0.07\}$), then refine around the best unperturbed/corrupted candidates. The coarse grid is a screening step, not a guarantee of locating the exact point-best `std_max`.

6.5 Implications for adaptive predictive-dynamics objectives

Read together, the evidence points at a method target rather than a finished algorithm. Input-side Gaussian augmentation is a coarse global scalar pressure: it helps, but it cannot separate nuisance variation from action-relevant variation, and the heteroscedastic-reweighting result below shows that gating by *prediction error* discards exactly the contact transitions that determine the next action. The natural next step is *adaptive predictive-dynamics consistency*: regularise the action-conditioned predictions of paired unperturbed/corrupted branches under the same action sequence — $F^k(z, \mathbf{a})$ against $F^k(\tilde{z}, \mathbf{a})$, the ACPC$_H$ objective of Section 3 — rather than the encoder outputs, and gate the regularisation by action/transition sensitivity (clean transition magnitude, inverse-dynamics signal, contact/keyframe cues) so that low-sensitivity transitions are pulled together while high-sensitivity transitions are guarded by the discriminability term of Equation (5). The operating principle is: low action sensitivity $\rightarrow$ stronger consistency; high action sensitivity $\rightarrow$ weaker consistency pull and stronger discriminability; high prediction difficulty alone should never trigger downweighting. We do not evaluate such an objective here; its immediate prerequisite is the paired-diagnostic study described next.

6.6 Limitations and future directions

Limitation 1 — Two backbone families validated; broader JEPA variants remain open. The toolkit and the predictive-consistency framing are introduced on LeWM; the PLDM sweep in Section F replicates the task-level signatures and the partial-correlation null on PushT. Section F also reports the PLDM full-diagnostic check, which exposes an important mechanism boundary: PLDM recovery checkpoints largely preserve rank, transition-resolution, and inverse-dynamics-probe statistics while reducing multi-step predictor drift. We therefore keep the main compression-chain interpretation LeWM-centred rather than claiming it as a universal mechanism. EMA-target JEPA variants (I-JEPA / V-JEPA lineage) and variational JEPA may exhibit different noise responses; we did not run them.

Limitation 2 — Gaussian pixel noise is the controlled training axis. The main sweep uses Gaussian pixel noise because it gives a scalar `std_max` axis for unperturbed/corrupted perturbation analysis. Section G adds a no-noise-checkpoint blur stress test and shows a corruption-specific profile: pixels+goal blur primarily collapses TwoRoom on both LeWM and PLDM (-63.67 pt and -68.33 pt), while PushT, Cube, and PLDM-Reacher are comparatively stable and LeWM-Reacher degrades mainly at the strongest blur endpoint. Visual fragility therefore generalises beyond the Gaussian-noise axis, but the task ordering and recovery profile are corruption-specific; blur-specific training and recovery remain follow-up work.

Limitation 3 — Diagnostic framework is empirical, not theoretical. Reacher and TwoRoom fail the partial-correlation criterion for *all* metrics, exposing where the empirical framework breaks down. We do not have a closed-form theory that predicts which tasks support a cross-checkpoint diagnostic and which do not; the current framework is a label-free *screen* with the empirical scope conditions in Section 6.3. Whether the residual checkpoint-quality signal we observe on PushT extrapolates to longer-horizon or out-of-grid `std_max` regions remains an open question.

Limitation 4 — The paired action-conditioned diagnostics remain to be computed. The ACPC-1, ACPC-H, predictive-cost-consistency (PCC), ranking-agreement (CRA), discriminability-margin (ADM), and SPRR quantities of Section 3.2 are protocol definitions for the next diagnostic pass; they are not part of the reported empirical evidence. They should be computed on the existing checkpoints with fixed unperturbed/corrupted observation pairs and a candidate action set held identical across branches, then tested against corrupted success and the corruption gap. The residual multi-step signal on Reacher (Section 5.5) is an encouraging precursor, but promoting the consistency objective (Section 6.5) to a method contribution requires this paired-diagnostic study and a subsequent training experiment.

Additional ablation — automated transition reweighting is not a substitute for noise training. As a sanity check we also tested a scale-preserving heteroscedastic negative-log-likelihood (NLL) formulation in which a learned per-transition σ-head down-weights high-error transitions. The result is informative as a negative finding: TwoRoom unperturbed success reaches 99.67% but high-noise robustness lags noise training; PushT unperturbed success *collapses* from 86.33% to

13.33% because contact-control transitions have high prediction error and are exactly the transitions σ-weighting would discard. Full data are reported in Section D. This is the sharpest expression of the discriminability side of action-conditioned predictive consistency: a global compression pressure that does not distinguish nuisance variation from action-relevant variation collapses the latter precisely where it is most needed ("hard $\neq$ nuisance" in contact control). It is also why a consistency objective must be gated by action sensitivity rather than by prediction error (Section 6.5); preserving action-relevant transitions instead of discarding them is the methodological direction left to follow-up work.

Future direction 1 — Adaptive predictive-dynamics consistency. The objective sketched in Section 6.5 — paired unperturbed/corrupted ACPC_H regularisation gated by action sensitivity, with an explicit discriminability guard — is the method this study motivates. Sections 5.5 and 5.6 already single out multi-step predictor drift as the residual-carrying signal on some tasks; turning its paired, action-conditioned variant into a per-token consistency controller is the open methodological question under investigation.

Future direction 2 — Broader cross-architecture replication. PLDM gives one second-family replication. Frozen/pretrained visual-feature models and EMA-target JEPA variants remain the next useful external checks.

Future direction 3 — Theoretical grounding. Reformulating the consistency/discriminability tension in an information-bottleneck / rate-distortion language is an attractive direction; we did not pursue it because the empirical phenomenology may not yet be stable enough for formal modelling.

7 Conclusion

This paper reframes visual robustness for JEPA world-model control: rather than encoder-level latent invariance, robustness should be defined as *action-conditioned predictive consistency* — unperturbed and corrupted views of the same state may encode differently, but under the same history and action sequence they should induce consistent task-relevant predictions — coupled with a discriminability countercondition that keeps action-distinct transitions separable (Section 3). Using controlled Gaussian corruptions as a probe on LeWorldModel, with PLDM as a second-family replication, the evidence supports this reframing:

1. Severe visual-corruption fragility in JEPA world-model control. Without noise-aware training, PushT exhibits a near-collapse under Gaussian pixel noise and TwoRoom shows severe degradation; latent prediction alone does not confer visual-corruption robustness in this protocol.
2. Input-side noise augmentation closes much of the gap but acts as a coarse global scalar pressure with broad, task-dependent plateaus rather than a single jointly optimal level; within a task the unperturbed and robustness optima can dissociate.
3. Pointwise, single-step diagnostics are the wrong abstraction. After conditioning on `std_max`, the strongest single-step fragility ratio does not isolate the corruption gap (the null replicates on PLDM and on the joint sample), whereas multi-step predictor drift carries residual signal on some tasks — the precursor of the paired, action-conditioned consistency we propose. A five-layer diagnostic protocol attributes the LeWM recovery to a compression chain, but the PLDM full-diagnostic check makes that a method-specific mechanism rather than a universal claim, and a negative heteroscedastic-reweighting result shows that error-based downweighting discards action-relevant transitions ("hard $\neq$ nuisance").

This paper does not propose a new training algorithm. Its contribution is a problem reframing, a body of diagnostic evidence, and a concrete paired-diagnostic protocol for action-conditioned consistency (Section 3.2). Together they motivate adaptive predictive-dynamics consistency (Section 6.5): enforce consistency after the action-conditioned predictor, gated by action sensitivity, while preserving the distinctions that change future transitions.

Acknowledgements

The complete code and data are available at `https://github.com/qun-team/wm_exp`. We thank the LeWM authors for releasing the baseline framework on which this study builds.

References

[1] Guillaume Alain and Yoshua Bengio. Understanding intermediate layers using linear classifier probes. ICLR 2017 Workshop Track / arXiv preprint, 2017.

[2] Mahmoud Assran, Quentin Duval, Ishan Misra, Piotr Bojanowski, Pascal Vincent, Michael Rabbat, Yann LeCun, and Nicolas Ballas. Self-supervised learning from images with a joint-embedding predictive architecture. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.

[3] Mido Assran, Adrien Bardes, David Fan, Quentin Garrido, Russell Howes, Mojtaba Komeili, Matthew Muckley, Ammar Rizvi, Claire Roberts, Koustuv Sinha, Artem Zholus, Sergio Arnaud, Abha Gejji, Ada Martin, Francois Robert Hogan, Daniel Dugas, Piotr Bojanowski, Vasil Khalidov, Patrick Labatut, Francisco Massa, Marc Szafraniec, Kapil Krishnakumar, Yong Li, Xiaodong Ma, Sarath Chandar, Franziska Meier, Yann LeCun, Michael Rabbat, and Nicolas Ballas. V-jepa 2: Self-supervised video models enable understanding, prediction and planning. *arXiv preprint arXiv:2506.09985*, 2025.

[4] Adrien Bardes, Quentin Garrido, Jean Ponce, Xinlei Chen, Michael Rabbat, Yann LeCun, Mahmoud Assran, and Nicolas Ballas. Revisiting feature prediction for learning visual representations from video (v-jepa). *Transactions on Machine Learning Research / arXiv:2404.08471*, 2024.

[5] Adrien Bardes, Jean Ponce, and Yann LeCun. Vicreg: Variance-invariance-covariance regularization for self-supervised learning. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2022.

[6] Ting Chen, Simon Kornblith, Mohammad Norouzi, and Geoffrey Hinton. A simple framework for contrastive learning of visual representations. In *Proceedings of the International Conference on Machine Learning (ICML)*, 2020.

[7] Ekin D. Cubuk, Barret Zoph, Jonathon Shlens, and Quoc V. Le. Randaugment: Practical automated data augmentation with a reduced search space. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) Workshops*, pages 702–703, 2020.

[8] Tom Dupuis, Jaonary Rabarisoa, Quoc-Cuong Pham, and David Filliat. VIBR: Learning view-invariant value functions for robust visual control. In *Proceedings of The 2nd Conference*

on Lifelong Learning Agents, volume 232 of *Proceedings of Machine Learning Research*, pages 658–682. PMLR, 2023.

[9] T. W. Epps and Lawrence B. Pulley. A test for normality based on the empirical characteristic function. *Biometrika*, 70(3):723–726, 1983.

[10] Carlos E. García, David M. Prett, and Manfred Morari. Model predictive control: Theory and practice — a survey. *Automatica*, 25(3):335–348, 1989.

[11] Quentin Garrido, Randall Balestriero, Laurent Najman, and Yann LeCun. Rankme: Assessing the downstream performance of pretrained self-supervised representations by their rank. In *Proceedings of the International Conference on Machine Learning (ICML)*, 2023.

[12] Carles Gelada, Saurabh Kumar, Jacob Buckman, Ofir Nachum, and Marc G. Bellemare. DeepMDP: Learning continuous latent space models for representation learning. In *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pages 2170–2179. PMLR, 2019.

[13] Hafez Ghaemi, Eilif Benjamin Muller, and Shahab Bakhtiari. seq-jepa: Autoregressive predictive learning of invariant-equivariant world models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.

[14] Christopher Grimm, Andre Barreto, Satinder Singh, and David Silver. The value equivalence principle for model-based reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, 2020.

[15] Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. Mastering diverse control tasks through world models. *Nature*, 640:647–653, 2025.

[16] Nicklas Hansen, Hao Su, and Xiaolong Wang. Td-mpc2: Scalable, robust world models for continuous control. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2024.

[17] Nicklas Hansen and Xiaolong Wang. Generalization in reinforcement learning by soft data augmentation. In *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, 2021.

[18] Yongchao Huang. Vjepa: Variational joint embedding predictive architectures as probabilistic world models. *arXiv preprint arXiv:2601.14354*, 2026.

[19] Li Jing, Pascal Vincent, Yann LeCun, and Yuandong Tian. Understanding dimensional collapse in contrastive self-supervised learning. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2022.

[20] David Klindt, Yann LeCun, and Randall Balestriero. When does lejepa learn a world model?, 2026.

[21] Simon Kornblith, Mohammad Norouzi, Honglak Lee, and Geoffrey Hinton. Similarity of neural network representations revisited. In *Proceedings of the International Conference on Machine Learning (ICML)*, 2019.

[22] Dirk P. Kroese, Sergey Porotsky, and Reuven Y. Rubinstein. The cross-entropy method for continuous multi-extremal optimization. *Methodology and Computing in Applied Probability*, 8(3):383–407, 2006.

[23] Yann LeCun. A path towards autonomous machine intelligence. OpenReview preprint, 2022.

[24] Lucas Maes, Quentin Le Lidec, Dan Haramati, Nassim Massaudi, Damien Scieur, Yann LeCun, and Randall Balestriero. stable-worldmodel-v1: Reproducible world modeling research and evaluation. ICLR 2026 Workshop on World Models (Tiny Paper); arXiv:2602.08968, 2026. `https://openreview.net/forum?id=wjMSWSPNco`.

[25] Lucas Maes, Quentin Le Lidec, Damien Scieur, Yann LeCun, and Randall Balestriero. Leworldmodel: Stable end-to-end joint-embedding predictive architecture from pixels. *arXiv preprint arXiv:2603.19312*, 2026.

[26] Mingyu Park, Samyeul Noh, Hyun Myung, and Donghwan Lee. Zero-shot visual generalization in model-based reinforcement learning via latent consistency. OpenReview (ICLR 2026 submission), 2025.

[27] Yuping Qiu, Rui Zhu, and Ying-cong Chen. Improving joint embedding predictive architecture with diffusion noise (n-jepa). *arXiv preprint arXiv:2507.15216*, 2025.

[28] Ashwath Radhachandran, Vedrana Ivezić, Shreeram Athreya, Ronit Anilkumar, Corey W. Arnold, and William Speier. Us-jepa: A joint embedding predictive architecture for medical ultrasound. *arXiv preprint arXiv:2602.19322*, 2026.

[29] Olivier Roy and Martin Vetterli. The effective rank: A measure of effective dimensionality. In *Proceedings of the European Signal Processing Conference (EUSIPCO)*, 2007.

[30] Yutaka Shimizu and Masayoshi Tomizuka. Bisimulation metric for model predictive control. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2025.

[31] Vlad Sobal, Jyothir S V, Siddhartha Jalagam, Nicolas Carion, Kyunghyun Cho, and Yann LeCun. Joint embedding predictive architectures focus on slow features, 2022.

[32] Vlad Sobal, Wancong Zhang, Kyunghyun Cho, Randall Balestriero, Tim G. J. Rudner, and Yann LeCun. Stress-testing offline reward-free reinforcement learning: A case for planning with latent dynamics models. In *WRL@ICLR 2025*, 2025.

[33] Yiyou Sun, Yifei Ming, Xiaojin Zhu, and Yixuan Li. Out-of-distribution detection with deep nearest neighbors. In *Proceedings of the International Conference on Machine Learning (ICML)*, pages 20827–20840, 2022.

[34] Alex Tamkin, Margalit Glasgow, Xiluo He, and Noah Goodman. Feature dropout: Revisiting the role of augmentations in contrastive learning. In *Proceedings of NeurIPS*, 2023.

[35] Jayden Teoh, Manan Tomar, Kwangjun Ahn, Edward S. Hu, Tim Pearce, Pratyusha Sharma, Akshay Krishnamurthy, Riashat Islam, Alex Lamb, and John Langford. Next-latent prediction transformers learn compact world models. *arXiv preprint arXiv:2511.05963*, 2025.

[36] Leonardo F. Toso, Davit Shadunts, Yunyang Lu, Nihal Sharma, Donglin Zhan, Nam H. Nguyen, and James Anderson. Learning invariant visual representations for planning with joint-embedding predictive world models. *arXiv preprint arXiv:2602.18639*, 2026.

[37] Hugues Van Assel, Mark Ibrahim, Tommaso Biancalani, Aviv Regev, and Randall Balestriero. Joint embedding vs reconstruction: Provable benefits of latent space prediction for self-supervised learning, 2025.

[38] Claas A. Voelcker, Anastasiia Pedan, Arash Ahmadian, Romina Abachi, Igor Gilitschenski, and Amir-Massoud Farahmand. Calibrated value-aware model learning with probabilistic environment models. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 61745–61768. PMLR, 2025.

[39] Tongzhou Wang and Phillip Isola. Understanding contrastive representation learning through alignment and uniformity on the hypersphere. In *Proceedings of the International Conference on Machine Learning (ICML)*, 2020.

[40] Denis Yarats, Rob Fergus, Alessandro Lazaric, and Lerrel Pinto. Mastering visual continuous control: Improved data-augmented reinforcement learning. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2022.

[41] Denis Yarats, Ilya Kostrikov, and Rob Fergus. Image augmentation is all you need: Regularizing deep reinforcement learning from pixels. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2021.

[42] Amy Zhang, Rowan T. McAllister, Roberto Calandra, Yarin Gal, and Sergey Levine. Learning invariant representations for reinforcement learning without reconstruction. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2021.

[43] Junbo Zhang and Kaisheng Ma. Rethinking the augmentation module in contrastive learning: Learning hierarchical augmentation invariance with expanded views. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 16650–16659, 2022.

A Experimental details

A.1 Environments

- **PushT.** 2D continuous pushing; 20,000 expert episodes; $\sim$196 steps/episode; action dim 2 (direction + magnitude); 224×224 RGB.
- **TwoRoom.** 2D continuous navigation; 10,000 episodes; $\sim$92 steps; action dim 2; 224×224 RGB.
- **Reacher.** 2D arm control (DeepMind Control Suite); 10,000 episodes; 200 steps; action dim 2.
- **Cube.** 3D cube manipulation (OGBench); 10,000 episodes; 200 steps; action dim 7.

A.2 Noise augmentation implementation

The actual implementation is `utils.py::AddNormalizedGaussianNoise`. Key points: (i) noise is added on ImageNet-normalized tensors and must be divided by the per-channel std to retain "pixel-space std" semantics; (ii) sampling is per-frame independent over the leading frame dims, not per-batch; (iii) all sweeps in this paper fix `noise_prob` = 1.0 and `std_min` = 0 and vary only `std_max`.

```
class AddNormalizedGaussianNoise:
    """Per-frame independent. Each frame draws Bernoulli(noise_prob) then
    std ~ Uniform(std_min, std_max). Pixel-space std is converted to
    normalized space by dividing by per-channel std before adding."""
    def __init__(self, std_min, std_max, noise_prob=1.0):
        self.std_low, self.std_high, self.noise_prob = std_min, std_max, noise_prob
        self.channel_std = torch.as_tensor(IMAGENET_STD) # (C,)
```

```python
    def __call__(self, x):
        if self.std_high <= 0 or self.noise_prob <= 0:
            return x
        leading = x.shape[:-3]
        stds = torch.empty(leading, device=x.device, dtype=x.dtype).uniform_(
            self.std_low, self.std_high)
        if self.noise_prob < 1.0:
            mask = (torch.rand(leading, device=x.device) < self.noise_prob).to(x.dtype)
            stds = stds * mask
        per_frame_scale = stds.view(*leading, 1, 1, 1)
        channel_factor = (1.0 / self.channel_std.to(x.device, x.dtype)).view(
            *([1] * len(leading)), -1, 1, 1)
        scale = per_frame_scale * channel_factor # pixel-space std -> normalized
        return x + torch.randn_like(x) * scale
```

A.3 Evaluation protocol

Unperturbed evaluation uses no noise, 100 trajectories per seed $\times$ 3 seeds (42/43/44), totalling 300 trajectories per condition. Noisy evaluation adds Gaussian noise of std $\in \{0.05, 0.08\}$ to both pixels and the goal image under the same 3-seed protocol. Success criterion is task-specific (PushT: T-block pose match; TwoRoom: target region; Reacher: joint-angle match; Cube: cube position match).

A.4 Compute

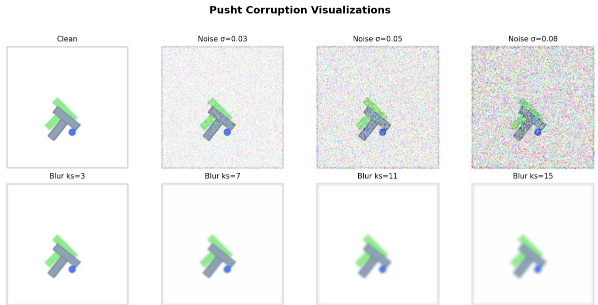
Training runs on a single NVIDIA H800 (80 GB) and follows the LeWM training schedule released with the baseline.

A.5 Main-figure rendering

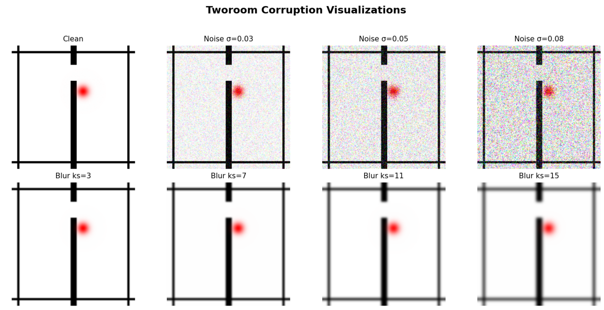
The five main figures are produced by `tools/paper1_figs.py`; run `python-mtools.paper1_figs--out-dirassets/paper1_figs`. The script reads `assets/paper1_data/canonical_evals_20260517.json` together with `assets/paper1_data/canonical_diagnostics_20260517.json`; no checkpoint or model loading is required.

A.6 Corruption visualizations

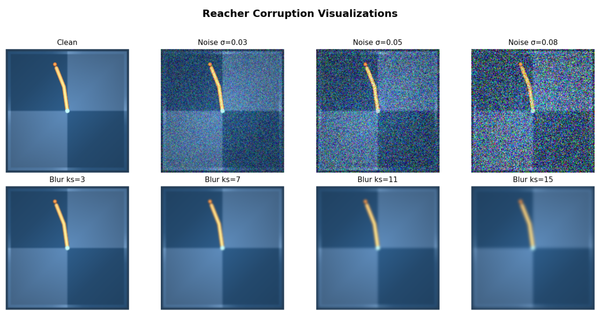
Figure 6 shows representative renderings of the two evaluated corruption families, applied to a sample observation from each of the four tasks. The Gaussian-noise row includes a mild calibration severity $\sigma = 0.03$ in addition to the two evaluation endpoints $\sigma \in \{0.05, 0.08\}$ used in Sections 5.2 and 5.3 (the main-sweep px+goal grid); the Gaussian-blur kernel sizes ($k \in \{3, 7, 11, 15\}$) match the stress test in Section G. Each panel is a single 224×224 RGB frame from the corresponding task's evaluation rollout, after corruption and before encoder normalisation.



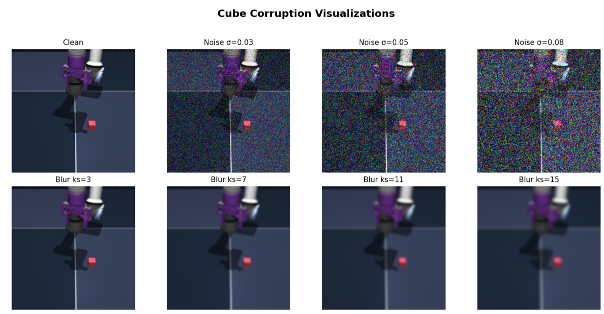
(a) PushT



(b) TwoRoom



(c) Reacher



(d) Cube

Figure 6: Corruption visualizations on the four tasks. Each task panel is a 2×4 grid: **Row 1** additive Gaussian pixel noise (the *Clean* reference and $\sigma \in \{0.03, 0.05, 0.08\}$); **Row 2** Gaussian blur at kernel sizes $k \in \{3, 7, 11, 15\}$ (sigma derived via the OpenCV / torchvision auto rule). $\sigma = 0.08$ is a strong stress-test endpoint at the 224×224 native input resolution; Table 1 reports the no-noise-training control success rate at this endpoint, and Tables 2 and 3 report the recovery achievable under noise-augmented training.

B Full diagnostic profile of LeWM-base on four tasks

Table 10 summarises every core diagnostic on the four LeWM-base checkpoints. Data are taken from each checkpoint's per-tool JSON outputs under `eval_results/diagnostics/` (the `geometry`, `task`, `predictor` and `latent_noise` families).

Table 10: Full LeWM-base diagnostics across four tasks.

Layer / Metric	TwoRoom	PushT	Reacher	Cube	Unit / note
Encoder geometry					
`clean_nn_cos_dist_median`	0.0449	0.2360	0.0633	0.1856	cosine distance
`clean_pair_cos_dist_median`	0.9904	1.0228	1.0252	1.0193	pair-wise cos dist
`clean_effective_rank`	47.60	76.42	61.04	73.25	effective rank
Noise sensitivity					
`noise_angle_deg_median` (@std= 0.05)	5.51°	1.33°	3.22°	1.40°	median angular shift
`noise_to_nn_cos_ratio_median`	0.1031	0.011	0.0249	0.016	noise/NN cos ratio
`robust_radius_std`	0.0142	0.0537	0.0142	0.0356	critical noise std
`first_risk_std`	>0.08	>0.08	>0.08	>0.08	first high-risk std
`noise_angle_slope_deg_per_std`	1085.8	284.8	831.7	327.0	°/std angular gain
`geometry_flag`	balanced	robust	balanced	robust	geometry label
Task resolution					
`transition_resolution_ratio_l2`	0.7216	0.3015	0.3704	0.4847	L2 resolution ratio
`transition_resolution_ratio_cos`	0.5538	0.0868	0.1351	0.2347	cos resolution ratio
`id_probe_r2`	0.2889	0.7739	0.1621	0.6657	linear probe R^2
`id_probe_r2_min`	0.2599	0.6786	0.1366	0.0972	min probe R^2
`lidar_rank`	46.06	13.95	45.90	42.46	LiDAR rank proxy
Action effect					
`action_mean_pred_shift_norm`	0.5329	0.1283	0.2518	0.2364	action-perturb mean shift
`action_perturb_pred_shift_corr`	0.2847	0.2873	0.4042	0.2559	shift $\times$ $\|a\|$ corr
Predictor stability					
`predictor_rollout_T8_l2`	18.62	18.65	15.17	20.20	T=8 rollout L2 drift
`predictor_target_to_nn_cos_ratio_at_max_std`	1.51e-4	3.54e-6	2.67e-5	3.39e-6	max std target/NN ratio
Latent noise					
`cka_linear_at_max_std`	0.1986	0.5536	0.3085	0.1814	CKA unperturbed vs noisy
`latent_cost_surface_slope_z`	635.31	1.3886	599.45	0.6208	goal latent cost slope

C Heteroscedastic-loss formulation

The scale-preserving heteroscedastic NLL referenced in Section 6.6 (the "Additional ablation" paragraph) and detailed in Section D is:

$$\mathcal{L}_{\text{hetero}} = \tfrac{1}{2}\,\exp(-s_t)\,\|z_{t+1} - \hat{z}_{t+1}\|^2 + \tfrac{1}{2}\,s_t\,, \tag{8}$$

where s_t is the σ-head-predicted log-variance. During training $\exp(-s_t)$ acts as an automatic weight: high-error transitions are down-weighted and low-error transitions are up-weighted. When $s_t \equiv 0$ the loss reduces to plain MSE.

D Heteroscedastic-loss negative result (data)

This appendix records the full data behind the "Additional ablation" paragraph in Section 6.6. The σ-head is trained jointly with the predictor; gradient is detached on the σ path so the mean prediction path is exactly LeWM MSE when σ is constant.

Table 11: Heteroscedastic-loss evaluation.

Task / model	Unperturbed	goal 0.05	pixels 0.05	px+goal 0.05	goal 0.08	px+goal 0.08
TwoRoom LeWM-base	94.00	73.33	72.33	61.33	58.67	50.00
TwoRoom LeWM+noise point-best	98.33	98.00	98.33	98.00	98.67	98.67
TwoRoom hetero	99.67	85.33	96.67	84.67	73.33	55.33
PushT LeWM-base	86.33	38.00	17.00	12.00	11.33	4.67
PushT LeWM+noise point-best	89.33	87.67	88.00	88.33	89.67	87.00
PushT hetero	13.33	7.67	7.67	7.67	9.67	6.00

TwoRoom hetero reaches 99.67% on unperturbed evaluation — consistent with the prior that low-dimensional discrete tasks benefit from stronger nuisance contraction / clustering — but lags noise training on high-noise robustness. **PushT hetero unperturbed success is** 13.33% **— a method-level failure**, not evidence for a useful robustness compromise.

Table 12: Representation diagnostics of the hetero-loss failure.

Metric	TwoRoom base	TwoRoom hetero	PushT base	PushT hetero
`clean_nn_cos_dist_median`	0.0449	0.0281	0.2360	0.1051
`clean_effective_rank`	47.60	33.59	76.42	42.85
`transition_resolution_ratio_`_cos	0.5538	0.3780	0.0868	0.0101
`transition_resolution_ratio_`_l2	0.7216	0.6055	0.3015	0.1023
`id_probe_r2`	0.2889	−0.0573	0.7739	0.2678
`action_mean_pred_shift_norm`	0.5329	0.4482	0.1283	0.0841
`predictor_rollout_T8_l2`	18.62	17.90	18.65	14.01

Hetero-loss compresses the representation in both tasks. For TwoRoom (low-dimensional, discrete, redundant), compression is acceptable. For PushT, the L2 transition-resolution ratio collapses from 0.3015 to 0.1023 and `id_probe_r`^{2} from 0.7739 to 0.2678 — task-relevant state information is erased. The drop in multi-step rollout drift is not good news either: the latent has become more *predictable* without being more *controllable*.

Mechanism. The σ-head correctly learns per-transition difficulty (high σ on contact frames in PushT, on doorway crossings in TwoRoom, etc.), but using σ as an automatic weight to down-weight high-error transitions misclassifies the contact-and-control-critical states of PushT as "unimportant" and erases them. This is a direct demonstration of the broader selective-consistency lesson: in contact-heavy control, **hard $\neq$ unimportant**.

This finding motivates a follow-up direction we are currently exploring: per-token *consistency* (not loss-reweighting) controllers driven by detached difficulty signals, which preserve the mean prediction path's gradient distribution. That work is independent of this paper's diagnostic study and will be reported separately.

E One-off cost-swap sanity check

This appendix records the eval-only cost-swap ablation referenced in Section 5.6.1. It is **not** part of the 36-checkpoint canonical evaluation table and uses a separate unperturbed reference (num_eval = 300), so we report it only as a sanity check rather than a pooled statistic.

Table 13: One-off eval-only cost swap on a TwoRoom checkpoint, std = 0.03 pixels+goal. Reference row reports a separate unperturbed eval of the same checkpoint (num_eval = 300).

Variant	Cost type	Cost space	Success rate
A (default)	cosine	normalized	36.0
B (swap)	mse	raw	42.0
Unperturbed reference (same ckpt)	—	—	69.7

Swapping cost recovers only +6 pt ($36 \to 42$), still far below the unperturbed reference (69.7). The narrow reading is therefore: **a sanity check suggests the cost function alone is unlikely to explain the visual-corruption collapse.**

F Cross-method replication: PLDM noise sweep on four tasks

This appendix records the second method-family on which the noise sweep and the cross-checkpoint correlation analysis replicate. PLDM follows the joint-embedding predictive line from Sobal et al. [31]; the concrete latent-dynamics planning baseline is from Sobal et al. [32]. We use the implementation distributed through the `stable-worldmodel` baseline suite [24]. Training and evaluation follow the LeWM canonical protocol bit-for-bit: per-frame Gaussian pixel noise via `utils.py::AddNormalizedGaussianNoise`, `std_max` $\in \{0.01, \ldots, 0.08\}$, three evaluation seeds (42/43/44), 100 trajectories per seed.

Coverage. The PLDM release is complete for the same grid as the LeWM canonical sweep: 4 tasks $\times$ 9 configurations (no-noise baseline plus `std_max` $\in \{0.01, \ldots, 0.08\}$), three evaluation seeds (42/43/44), and 100 trajectories per seed. The PLDM eval, predictor-diagnostic, full-diagnostic, and cross-method-correlation JSON files are listed in the data manifest; we do *not* merge them into the LeWM canonical files used by Tables 1–7.

No-noise-trained PLDM cliff. PLDM reproduces the no-noise-trained visual-corruption cliff on PushT and TwoRoom, while Reacher and Cube show much smaller drops under this PLDM training recipe (Table 14). This supports a method-family reading of the failure mode without claiming every task has the same cliff magnitude.

Table 14: PLDM baseline trained without input-noise augmentation on all four tasks. Success rate mean $\pm$ population std, 3 seeds $\times$ 100 trajectories.

Task	unperturbed	px+goal 0.05	px+goal 0.08	unperturbed $\to$ 0.08 drop
TwoRoom	96.67 ± 1.70	68.33 ± 3.30	51.67 ± 2.05	-45.00
PushT	75.33 ± 3.68	43.67 ± 4.64	10.00 ± 2.16	-65.33
Reacher	82.67 ± 1.70	82.33 ± 0.94	79.33 ± 0.94	-3.33
Cube	52.67 ± 3.30	51.33 ± 2.49	48.33 ± 2.87	-4.33

PLDM full sweep — unperturbed evaluation. Table 15 reports the per-task unperturbed success rate at every trained `std_max` level. Table 16 reports the corresponding px+goal 0.08 robustness.

Table 15: PLDM+noise sweep, unperturbed (clean condition-name) success rate (%). † marks the per-task unperturbed point-best.

`std_max`	TwoRoom	PushT	Reacher	Cube
0 (base)	96.67 ± 1.70	75.33 ± 3.68	82.67 ± 1.70	52.67 ± 3.30
0.01	96.33 ± 0.94	72.33 ± 4.19	78.00 ± 0.82	51.33 ± 1.25
0.02	97.33 ± 0.47	71.33 ± 3.86	82.33 ± 2.36	53.33 ± 1.70
0.03	96.67 ± 1.70	$76.67 \pm 7.54^{\dagger}$	83.00 ± 3.74	52.67 ± 3.30
0.04	97.67 ± 0.47	68.67 ± 5.25	$85.00 \pm 2.16^{\dagger}$	54.00 ± 2.94
0.05	97.67 ± 1.25	74.33 ± 3.40	72.00 ± 1.41	54.00 ± 2.16
0.06	98.00 ± 0.82	70.33 ± 6.18	79.00 ± 0.82	$56.00 \pm 4.97^{\dagger}$
0.07	98.00 ± 0.82	66.67 ± 8.38	80.67 ± 2.62	53.67 ± 3.68
0.08	$98.33 \pm 0.94^{\dagger}$	68.00 ± 1.41	80.33 ± 1.25	54.00 ± 1.63

Table 16: PLDM+noise sweep, px+goal 0.08 success rate (%). ‡ marks the per-task px+goal 0.08 point-best.

`std_max`	TwoRoom	PushT	Reacher	Cube
0 (base)	51.67 ± 2.05	10.00 ± 2.16	79.33 ± 0.94	48.33 ± 2.87
0.01	89.33 ± 3.30	16.67 ± 4.03	78.00 ± 1.63	53.00 ± 1.63
0.02	93.33 ± 0.94	40.00 ± 0.82	79.33 ± 2.05	52.33 ± 3.40
0.03	94.33 ± 1.70	$72.00 \pm 2.94^{\ddagger}$	79.33 ± 2.36	55.67 ± 4.11
0.04	98.00 ± 0.00	62.33 ± 7.41	$81.33 \pm 1.70^{\ddagger}$	53.00 ± 3.56
0.05	$98.33 \pm 0.94^{\ddagger}$	69.33 ± 4.92	59.67 ± 3.30	53.00 ± 1.63
0.06	97.33 ± 1.25	69.67 ± 5.25	76.00 ± 1.41	53.33 ± 3.09
0.07	96.67 ± 0.47	67.00 ± 9.09	77.67 ± 3.30	52.00 ± 0.00
0.08	97.00 ± 0.82	66.33 ± 4.78	80.67 ± 2.36	$56.33 \pm 1.25^{\ddagger}$

Reading. Four quantitative observations are useful for interpreting PLDM as a second method family:

1. **TwoRoom (visually redundant) benefits from heavy noise on both methods.** PLDM px+goal 0.08 success rises from 51.67% at the no-noise baseline to 98.33% at `std_max` $= 0.05$, then remains on a high-noise plateau. This mirrors the LeWM reading that TwoRoom tolerates strong same-state nuisance contraction.
2. **PushT (contact-heavy) exhibits the corruption cliff and large noise-training recovery.** PLDM without noise augmentation drops by 65.33 pt under px+goal 0.08; training with `std_max` $=$ 0.03 recovers px+goal 0.08 success to 72.00% (+62.00 pt over the no-noise baseline). The unperturbed and px+goal 0.08 point-bests are co-located at `std_max` $= 0.03$ on PLDM, unlike LeWM where the robust point-best moves to a heavier-noise checkpoint.
3. **Reacher is already robust under no-noise-trained PLDM.** The no-noise PLDM drop is only 3.33 pt, and the point-best for both unperturbed and px+goal 0.08 sits at `std_max` $= 0.04$ (85.00% unperturbed, 81.33% px+goal 0.08). We therefore treat Reacher as a low-cliff external baseline rather than evidence for a universal noise-training gain.
4. **Cube (structured 3D manipulation) remains weakly noise-responsive.** PLDM Cube unperturbed success spans 51.33–56.00% and px+goal 0.08 spans 48.33–56.33% across the full grid. This is the same weak noise responsiveness seen on LeWM.

Method-specific details. PLDM's PushT unperturbed point-best (76.67% at `std_max` $= 0.03$) is lower than LeWM's (89.67% at `std_max` $= 0.03$), so the external baseline is not a performance leaderboard. It is a robustness check: the cliff-and-recovery shape is reproduced, while the exact unperturbed/robust optimum location is method-dependent. In particular, the LeWM PushT optimum dissociation should be read as a LeWM-family signature, not a cross-method law.

Cross-method partial correlations. We repeat the fragility ratio partial-correlation analysis for PLDM on all four tasks. The headline PushT null replicates in the limited sense that we do not see stable evidence for a non-zero residual corruption-gap association: PLDM has unconditional $\rho(\text{metric}, \text{corruption drop}) = -0.78$ but partial $\rho(\text{metric}, \text{corruption drop}) \mid \texttt{std\_max} = -0.14$ (95% bootstrap CI $[-1.00, +0.87]$), and the joint LeWM+PLDM PushT partial after conditioning on `std_max` and method is $+0.11$ (95% CI $[-0.54, +0.71]$). The intervals are wide and include zero. Other tasks show task-specific residuals, so we use the full table as a scope boundary rather than as a universal diagnostic claim.

Table 17: PLDM fragility ratio partial correlations, $n = 9$ per task. The first three partial columns condition on `std_max` within PLDM; the final column uses the joint LeWM+PLDM sample ($n = 18$) and conditions on both `std_max` and a method dummy. Numbers come from the released PLDM correlation JSON.

Task	PLDM n	unperturbed partial	px+goal 0.08 partial	corruption-drop partial	joint corruption-drop partial
TwoRoom	9	−0.26	−0.85	+0.87	+0.56
PushT	9	−0.22	+0.04	−0.14	+0.11
Reacher	9	−0.68	−0.86	+0.14	+0.34
Cube	9	−0.36	−0.05	−0.41	−0.13

PLDM full-diagnostic mechanism check. We also run the same five diagnostic layers on the PLDM sweep and release the resulting per-checkpoint summary in `canonical_full_diagnostics_pldm_20260523.json`. The compact base-vs-representative view in Table 18 shows a different mechanism profile from the LeWM compression chain in Table 4: PLDM's representative recovery checkpoints do *not* exhibit a broad collapse in rank, transition resolution, or inverse-dynamics probe. The most consistent movement is instead a reduction in multi-step predictor drift. This is useful precisely because it separates two claims: the task-level fragility/recovery signature replicates across method families, while the exact diagnostic mechanism is architecture-dependent.

Table 18: PLDM full-diagnostic check, no-noise baseline → representative noise-trained checkpoint. Representatives are each task's PLDM px+goal 0.08 point-best ckpt (TwoRoom `std_max` = 0.05, PushT `std_max` = 0.03, Reacher `std_max` = 0.04, Cube `std_max` = 0.08). Values come from the released full-diagnostics JSON.

Task	representative `std_max`	effective rank	transition ratio L2	inverse-dynamics probe R^2	rollout T8 L2
TwoRoom	0.05	74.78 → 72.80	0.8188 → 0.8270	0.3233 → 0.3273	20.36 → 1.19
PushT	0.03	130.13 → 131.90	0.4710 → 0.4778	0.7570 → 0.7479	20.25 → 2.82
Reacher	0.04	117.52 → 114.99	0.5053 → 0.5104	0.2150 → 0.1960	1.45 → 1.23
Cube	0.08	134.80 → 124.79	0.6395 → 0.6401	0.5428 → 0.5531	18.98 → 0.62

What this strengthens, and what it does not. PLDM strengthens C2 by showing that the four task signatures are not tied to one LeWM checkpoint family. It strengthens C4 only in the precise PushT sense stated in the main text: after removing the sweep-level `std_max` effect and the method offset, the local predictor-sensitivity diagnostic does not isolate corruption-specific robustness. The TwoRoom and Reacher PLDM residuals are deliberately left visible in Table 17. The TwoRoom corruption-drop partial = +0.87 does not contradict the headline null: both unperturbed and noisy TwoRoom success are saturated near 97–98% across the PLDM sweep, so the corruption-drop range is only a few points in absolute terms and the partial picks up rank-order noise inside that band rather than a controllable robustness signal. The Reacher px+goal 0.08 partial $= -0.86$ similarly reflects a tight noisy-eval cluster (59.67–81.33% across the sweep) and tracks noisy success rather than the corruption gap. Both residuals are task-specific artefacts of saturation and should not be converted into a robustness oracle.

Mechanism boundary. Table 18 is the reason we do not promote the LeWM compression chain to a universal theorem. On LeWM, noise-trained representatives can compress effective rank and reduce transition-key resolution; on PLDM, the same task-level robustness pattern is accompanied mainly by reduced rollout drift with largely preserved rank/resolution probes. The shared conclusion is therefore about *control-time visual fragility and noise-training recovery*; the internal diagnostic route is method-specific.

G Cross-corruption sanity check: no-noise-trained blur evaluation

This appendix addresses the narrow concern that the observed visual fragility is an artefact of Gaussian noise only. We evaluate the LeWM and PLDM baselines trained without input-noise augmentation under Gaussian blur with kernel sizes $3, 7, 11, 15$, applied to pixels, goal images, or both. This is an **eval-only** stress test: no blur-trained checkpoint is included, and the blur data are not mixed into the Gaussian-noise sweep. Source: `canonical_blur_baselines_20260523.json`.

Table 19: Pixels+goal blur stress test on baselines trained without input-noise augmentation. Success rate mean $\pm$ population std over 3 evaluation seeds $\times$ 100 trajectories. Kernel sizes 11 and 15 are severe low-pass endpoints on 224×224 inputs, so they should not be read as mild corruptions.

Method	Task	unperturbed	$k = 3$	$k = 7$	$k = 11$	$k = 15$	worst drop
LeWM	TwoRoom	94.00 ± 3.56	39.33 ± 2.05	32.67 ± 3.09	30.33 ± 2.87	30.67 ± 0.47	-63.67
LeWM	PushT	86.33 ± 2.36	81.67 ± 4.50	74.00 ± 1.41	65.33 ± 2.49	58.67 ± 6.65	-27.67
LeWM	Reacher	58.67 ± 1.25	57.00 ± 6.38	45.67 ± 2.05	36.00 ± 4.90	20.33 ± 2.49	-38.33
LeWM	Cube	66.67 ± 2.62	65.00 ± 1.63	62.33 ± 2.62	57.33 ± 2.87	54.00 ± 2.16	-12.67
PLDM	TwoRoom	96.67 ± 1.70	38.33 ± 0.47	30.33 ± 2.87	28.33 ± 1.25	28.33 ± 1.70	-68.33
PLDM	PushT	75.33 ± 3.68	74.00 ± 2.94	72.00 ± 3.56	67.67 ± 4.03	62.33 ± 2.05	-13.00
PLDM	Reacher	82.67 ± 1.70	86.00 ± 2.16	80.00 ± 3.56	72.00 ± 2.16	68.00 ± 4.08	-14.67
PLDM	Cube	52.67 ± 3.30	53.00 ± 5.35	53.00 ± 2.83	50.67 ± 3.68	52.33 ± 5.44	-2.00

Reading. Blur is a different failure mode from Gaussian pixel noise. The clear collapse is TwoRoom: it falls below 40% already at $k = 3$ on both LeWM and PLDM and then saturates around 28–33% for $k = 7$–15. This reverses the Gaussian-noise ordering, where PushT is the most fragile task. The other tasks are more stable under blur: PLDM PushT, PLDM Reacher, and both Cube baselines lose at most about 15 pt at the strongest endpoint, while LeWM PushT degrades gradually and LeWM Reacher drops sharply only at $k = 15$. We therefore use blur only as an eval-only cross-corruption sanity check: visual fragility is not unique to Gaussian noise, but the task ordering and recovery profile are corruption-specific.